

Prague University of Economics and Business

Faculty of Informatics and Statistics



Test case design for the mobile application Pilulka “Pill”

BACHELOR THESIS

Study programme: Applied Informatics

Field of study: Applied Informatics

Author: Alesia Zhiliaeva

Supervisor: Ing. et Ing. Michal Doležel, Ph.D.

Prague, May 2022

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, Ing. et Ing. Michal Doležel Ph.D., for his patient guidance, valuable comments, and advice I was receiving throughout the whole process of writing this thesis. I would also like to thank my colleagues from Pilulka.cz for their cooperation and answers to the questions that I had.

Abstract

The aim of this thesis is to design test cases for the mobile application Pilulka by providing theoretical knowledge for designing test cases and applying it in the practical part of this thesis. The thesis is divided into seven chapters. The first chapter deals with software requirements. The second chapter summarizes test analysis methods and the design of test cases. The third chapter details the use of test management tools and outlines one of them. The fourth chapter is dedicated to the presentation of the tested mobile application and the analysis of selected requirements. The fifth chapter focuses on test cases using specification-based techniques. The sixth chapter contains the design of test cases derived from the functional requirements listed in this work. The last chapter demonstrates the practical application of test cases inside of the test management tool.

Keywords

Test cases, software requirements, mobile application, mobile application testing

Content

Introduction	8
1 Software requirements	10
1.1 Definition of requirements	10
1.2 Types of requirements	10
1.3 Requirements elicitation	13
1.4 Software requirements specification	13
1.4.1 Waterfall approach	14
1.4.2 Agile approach	15
1.5 Characteristics of good requirements	16
2 Test analysis methods	18
2.1 Test case design	18
2.1.1 Test idea	18
2.1.2 Test case	18
2.1.3 Test case writing guidelines	19
2.2 Specification-based techniques	20
2.2.1 Equivalence partitioning	20
2.2.2 Boundary value analysis	20
2.2.3 Decision table testing	21
2.2.4 State transition testing	22
2.3 Experience-based techniques	22
2.3.1 Exploratory testing	22
2.3.2 Ad hoc testing	23
2.3.3 Error guessing	23
3 Test management tools	24
3.1 TestRail	24
4 Requirements analysis of the tested application	29
4.1 Specification of application domain	29
4.2 Description of the application Pilulka	29
4.2.1 Profile	30
4.2.2 Settings	31
4.2.3 Shopping cart	31
4.2.4 Search	32
4.2.5 Catalogue	32

4.2.6 Pilulka Fit	33
4.3 Selected functional requirements	34
4.4 Traceability matrix.....	36
5 Application of specification-based testing techniques	38
5.1 Free delivery.....	38
5.2 Product review.....	38
5.3 Admission to the Pilulka Fit club	39
6 Test case design based on user stories	41
6.1 Login.....	41
6.2 Favourite products.....	42
6.3 Repeat order	44
6.4 EAN scanner.....	45
6.5 Substitution of products	46
6.6 Country of the application.....	47
7 Management of test cases in TestRail.....	50
7.1 Test organization	50
7.2 Test run and tests.....	51
7.3 Test results.....	52
Conclusion	54
List of references	55
Annexes.....	I
Annex A: Test case – Free delivery	I
Annex B: Test cases – Product review	II
Annex C: Test cases – Admission to the Pilulka Fit club.....	V

List of figures

Figure 1 3 levels of software requirements [Adapted from Wiegers & Beatty (2013, p. 12)]	11
Figure 2 FURPS quality attributes [Adapted from Doležel (2022)]	12
Figure 3 SRS template [Adapted from Wiegers & Beatty (2013, p. 12)]	14
Figure 4 SRS document preparation by the waterfall approach [Adapted from Robertson & Robertson (2012, str. 24)]	15
Figure 5 Preparation of user stories by the iterative approach [Adapted from Robertson & Robertson (2012, str. 24)]	15
Figure 6 Example of the state transition diagram [Source: own elaboration]	22
Figure 7 Features of TestRail [Source: TestRail, 2020]	25
Figure 8 Main dashboard [Source: TestRail, 2020]	26
Figure 9 Project dashboard [Source: TestRail, 2020]	26
Figure 10 Main sections [Source: TestRail, 2020]	26
Figure 11 Test Case Template [Source: TestRail, 2020]	27
Figure 12 Test Run [Source: TestRail, 2020]	28
Figure 13 Test statuses [Source: TestRail, 2020]	28
Figure 14 Profile screen	30
Figure 15 Log in screen	30
Figure 16 Settings	31
Figure 17 Shopping cart	32
Figure 18 Notifications	33
Figure 19 Catalogue	33
Figure 20 Weekly challenges	33
Figure 21 Step counter	33
Figure 22 State transitioning – Admission to the Pilulka Fit club [Source: own elaboration]	39
Figure 24 Imported test cases [Source: own elaboration]	50
Figure 25 The details of a test case: C3 - EAN scanning [Source: own elaboration]	51
Figure 26 Status of the test: C2 - Substitution of out-of-stock products [Source: own elaboration]	52
Figure 27 Statuses of test steps: C2 - Substitution of out-of-stock products [Source: own elaboration]	52
Figure 28 The overview of the test run [Source: own elaboration]	53

List of tables

Table 1 Example of the equivalence partitioning [Source: own elaboration]	20
Table 2 Example of the boundary value analysis [Source: own elaboration]	21
Table 3 Example of the decision table testing [Adapted from Hamilton (2022)]	21
Table 4 List of selected functional requirements [Source: own elaboration]	34
Table 5 Traceability matrix.....	37
Table 6 Equivalence partitioning and a boundary value analysis – Free Delivery [Source: own elaboration]	38
Table 7 The decision table - Product review [Source: own elaboration].....	39
Table 8 Table of state transitioning [Source: own elaboration]	40
Table 9 TC_04.01 – Login via email [Source: own elaboration]	41
Table 10 TC_04.02 – Login via Apple ID, Google Account and Facebook [Source: own elaboration]	42
Table 11 TC_05.01 – Favourite products for non-logged in users [Source: own elaboration]	43
Table 12 TC_05.02 – Favourite products for logged-in users [Source: own elaboration]..	43
Table 13 TC_06.01 – Repeat order with an empty cart [Source: own elaboration]	44
Table 14 TC_06.02 – Repeat order not with an empty cart [Source: own elaboration]	44
Table 15 TC_07.01 – EAN scanning [Source: own elaboration].....	45
Table 16 TC_08.01 – Substitution of non-available products [Source: own elaboration]..	46
Table 17 TC_08.02 – Substitution of out-of-stock products [Source: own elaboration]....	46
Table 18 TC_09.01 – Automatic set of the country [Source: own elaboration]	47
Table 19 TC_09.02 – Country change via settings [Source: own elaboration]	48
Table 20 TC_01.01 – Free delivery: boundary values [Source: own elaboration].....	I
Table 21 TC_02.01 – Submission of the product review: stars and text [Source: own elaboration]	II
Table 22 TC_02.02 – Submission of the product review: stars and a positive point [Source: own elaboration]	II
Table 23 TC_02.03 – Submission of the product review: stars and a negative point [Source: own elaboration]	III
Table 24 TC_02.04 – Submission of the product review: stars, negative and positive points [Source: own elaboration]	III
Table 25 TC_02.05 – Submission of the product review: stars, text, negative and positive points [Source: own elaboration].....	IV
Table 26 TC_02.06 – Submission of the product review: no stars and a positive point [Source: own elaboration]	IV
Table 27 TC_03.01 – Admission to the Pilulka Fit club: data allowed [Source: own elaboration]	V
Table 28 TC_03.02 – Admission to the Pilulka Fit club: data not allowed [Source: own elaboration]	V
Table 29 TC_03.03 – Admission to the Pilulka Fit club: data allowed via settings [Source: own elaboration]	VI
Table 30 TC_03.04 – Admission to the Pilulka Fit club: without logging in [Source: own elaboration]	VI

Introduction

According to the study conducted by the APP Annie (2021), nearly 218 billion mobile applications have been downloaded via the Play Store and App Store platforms during the year 2020. This number indicates that the applications have become a central part of the lives of billions of users who are becoming more and more demanding. With a few clicks, modern people can order needed medicine directly from home, book a hotel on the other side of the world, or make a video call to their friends. For every need and desire, there is already an application that can help get things done easily and conveniently, but to be truly successful, a mobile application must first be well tested.

A mobile user expects the application to be simple, intuitive, and work consistently at any place without crashing. If the expectations are not met, the user simply installs a similar app made by another author. Therefore, the quality of the application is one of the main factors of its popularity and competitiveness. Proper testing of mobile applications enables companies to provide better software and helps each application to succeed by verifying its functionality, usability, and performance, thereby expanding its user base. The design of test cases is one of the key factors of thorough testing and serves as the basis of identifying and covering the features of the mobile application that need to be tested.

Reason for topic selection

The main motivation for choosing this topic is the author's work experience as a tester at Pilulka.cz. One of the author's main responsibilities was to create test cases in order to regularly test and improve the quality of the software. Another reason for choosing this topic is the author's personal interest in testing and her wish to deepen the theoretical knowledge about designing test cases and testing mobile applications.

Aim and objectives

The aim of this bachelor thesis is to design test cases for the mobile application Pilulka "Pill". This goal is realized by fulfilling theoretical and practical objectives:

1. To provide the necessary theoretical knowledge for the design of test cases and to present testing methods used for the mobile applications.
2. To summarize the key functional requirements for the development of test cases.
3. To describe the specifics of the online pharmacy market.
4. To design test cases for the selected functional requirements of the application using the test analysis methods.
5. To demonstrate a work with test cases in the test management tool TestRail.

Overview and structure

The first part of the thesis deals with the theoretical background that is needed for the following practical part of this work. The overall theory is covered by Chapter 1, Chapter 2, and Chapter 3. These chapters include the description of software requirements and the activities related to working with them. In addition to that, theoretical part also focuses on the test case design techniques and specifics of test case writing. It clarifies the application of the test management tools and describes in detail how to use one of them.

The practical part of this thesis is devoted to the design of test cases for the mobile application Pilulka, based on the functional requirements presented in Chapter 4. This part introduces the specifics of the online pharmacy market, describes the Pilulka mobile application, and includes the analysis of the requirements. Chapter 5 deals with the test cases derived from the specification-based testing techniques, while Chapter 6 focuses on the test cases based on the functional requirements. The final chapter of this thesis demonstrates the management and execution of test cases using the test management tool TestRail.

Contribution

The main contribution of this thesis is the designed test cases for selected functional requirements of a mobile application Pilulka in order to help testers in this company to perform appropriate functional testing of the mobile application. Since the mobile testers at Pilulka.cz check the functionality of individual application components manually and their tests are not yet performed according to predefined scenarios, the test cases presented in this thesis can be a good demonstration of how to create and use test cases for mobile testing. In the future, the test cases designed in this work can also be used as a basis for creating automated tests.

1 Software requirements

One of the first tasks at the beginning of the software development process is the collection and analysis of software requirements. For this reason, this chapter **clarifies the importance of software requirements in product development and testing**. The first subsection 1.1 introduces the term "software requirements" from the perspective of various authors. The next subsection 1.2 deals with the classification of requirements. Subsection 1.3 explains the process of requirements elicitation. The following subsection 1.4 introduces the requirements specification phase and describes the process of documenting requirements in 2 different methodologies: Agile and Waterfall model. The last subsection 1.5 outlines the main qualities of the well-written software requirements.

1.1 Definition of requirements

The term "software requirements" is ambiguous and does not have a precise definition, as a result, individual authors and associations define this term differently, depending on their own experience and understanding. For instance, the classic definition of software requirements was given by the Institute of Electrical and Electronics Engineers (IEEE) in 1990 (IEEE Computer Society, p.62), where the term was defined as "*A condition or capability needed by a user to solve a problem or achieve an objective...*" A requirement is therefore a necessary condition for accessing the next step in the software development process. However, this definition is still very vague and far too imprecise.

The definition proposed by Ian Sommerville and Pete Sawyers and then cited by Wiegers & Beatty (2013, p. 29) is more precise: "*Requirements are a specification of what should be implemented. They are descriptions of how the system should behave, or of a system property or attribute. They may be a constraint on the development process of the system.*" This definition again suggests that a requirement "constrains" the development process according to the expectations of the external user (customer) and the internal user (developer). What is important is the role of the customer who determines the requirements that will frame the development process.

Such authors as Andrew Stellman and Jennifer Greene (2005) point out in their book, that software requirements serve as the basis for software development. According to these authors, requirements help in true understanding of how the software is supposed to behave before its build.

1.2 Types of requirements

The question of classifying types of requirements has been discussed by a number of authors. Some of them (Wiegers & Beatty, 2013) divide software requirements into three main groups: **business requirements, user requirements and functional**

requirements. Furthermore, Wiegers & Beatty (2013) state that every system has a set of non-functional requirements.

Nonetheless, this approach of distinguishing the requirements is not fully applicable in practice. According to the author's personal experience, the requirements are not usually defined and divided by different roles such as: corporate and commercial roles (see Figure 1). Besides that, since there are no fixed boundaries between business, user and functional requirements, this way of categorizing requirements is more of a theory than a common practical case. The level of software requirements used in this work, together with the stakeholders involved, is shown in Figure 1.

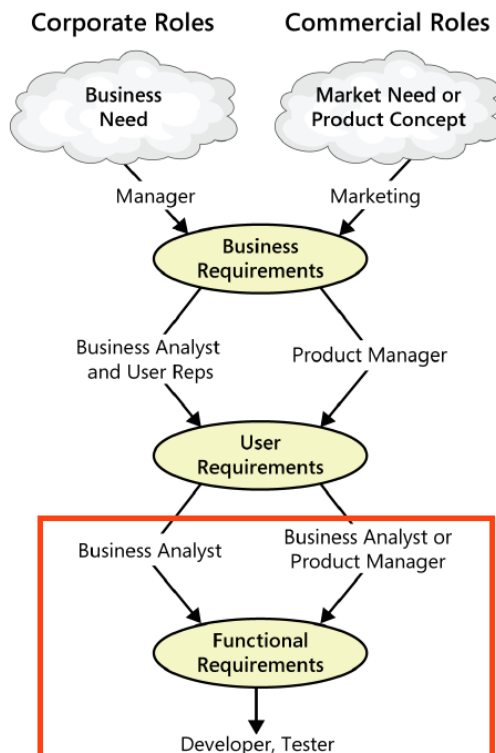


Figure 1 3 levels of software requirements [Adapted from Wiegers & Beatty (2013, p. 12)]

The binary distinction of software requirements is supported by the well-known FURPS model, which was developed at Hewlett-Packard in 1987 (Grady & Caswell, 1987). This method is based on 5 main requirements, distributed between functional requirements "F" and non-functional "URPS" requirements. Figure 2 shows 5 software quality attributes according to the FURPS method: **Functionality, Usability, Reliability, Performance and Supportability**.

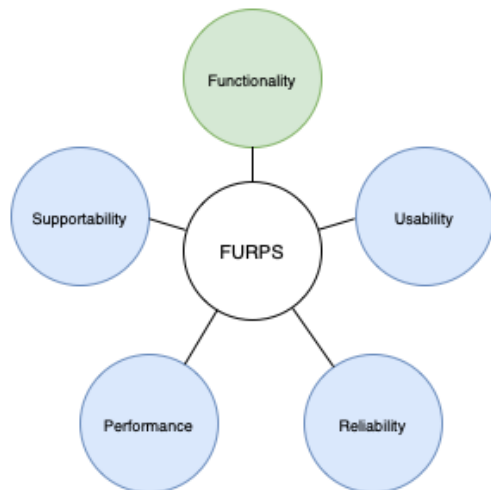


Figure 2 FURPS quality attributes [Adapted from Doležel (2022)]

Functional requirements

Functional requirements determine what the system has to achieve. The IEEE Computer society (1990, p. 35) defines a functional requirement as "*a function that a system or component must be able to perform*". These requirements describe how the software interacts with its environment and specify the inputs, outputs, external interfaces, and functions that the software should contain. Functional requirements are primarily aimed at providing a software solution to a customer that needs to satisfy its users, so specifying functional requirements is meaningless if it does not take into account user expectations.

Non-functional requirements

As stated by Chung et al. (2012), non-functional requirements play a critical role in system development. In contrast to functional requirements, non-functional requirements relate to security, reliability, usability, performance, compatibility, portability, etc. (Roudenský and Havlíčková, 2013). The aforementioned FURPS method, defines 4 non-functional requirements:

Usability refers to how the user perceives the system. The main criteria for usability are navigation, personalization, consistency, ease of use of the system, etc.

Reliability includes aspects such as availability, accuracy, and recoverability of the system. It indicates the probability that the software will function without errors.

Performance is related to the response time of the system. It means the amount of time it takes for the system to load, start, and recover.

Supportability can consist of a combination of different properties such as testability, adaptability, and compatibility.

1.3 Requirements elicitation

The practice of discovering and collecting different types of requirements explained in the previous subsection is called the requirements elicitation (Wiegiers & Beatty, 2013). This process is an essential part of any software development project. Gathering of requirements can be viewed as a process in which a requirements analyst collects, learns, reviews, and expresses the needs of the stakeholders and users of a software project (Stellman & Greene, 2005). The goal of this activity is to create a complete list of requirements that all groups of system users consider important.

Karl E. Wiegiers (2013, p. 19) thinks of the requirements elicitation as a more complex phase than the actual writing of requirements: *“Writing requirements is a matter of clarifying, elaborating, and recording what you’ve learned. A solid understanding of a product’s requirements ensures that your team works on the right problem and devises the best solution to that problem.”*

Before communicating with customers and users to obtain their requirements for the system, it is necessary to determine who will use the system and to identify other stakeholders of the product. After that, a variety of approaches can be used to gain a full understanding of user needs. However, it is important to note that requirements gathering is an iterative process and there is no single technique that is universally applicable.

The result of the requirements elicitation and analysis phase is the documentation of the collected requirements, which is regularly reviewed, clarified, and expanded as changes occur during the project.

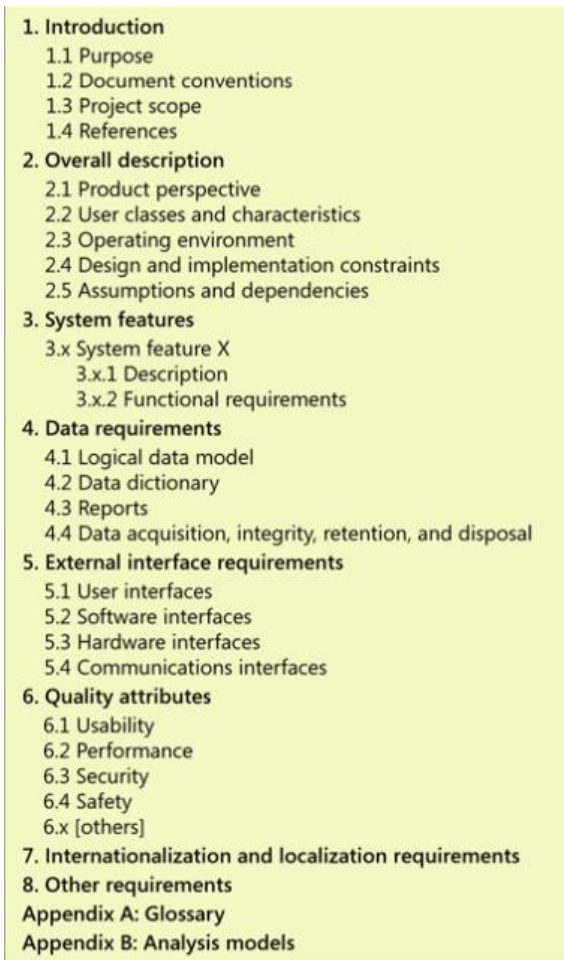
1.4 Software requirements specification

The phases of requirements elicitation, analysis, and documentation end with the completion of a document called the Software Requirements Specification (SRS) (Wiegiers & Beatty, 2013). This document is the basis from which developers get their assignments and QA engineers understand how to create test cases. As maintained by Roudenský a Havlíčková (2013), a software requirements specification contains functional and non-functional requirements in addition to a detailed description of the behaviour and features of the future system.

The preparation of the SRS must be handled with particular care, as this document forms the basis of the agreement between the client and the developer and later also serves as contract foundation between the client and its customers. According to Robertson and Robertson (2013), every software development organization should adopt one or more standard SRS templates for their projects. For example, Karl E. Wiegiers and Joy Beatty (2013) recommend an SRS template that is suitable for multiple types of projects, this template is presented below in Figure 3.

However, as maintained by Robertson and Robertson (2012), the level of formality and detail of the SRS varies depending on the methodology used (Agile vs. Waterfall).

Agile and traditional projects manage requirements differently in various aspects, especially regarding the timing of activities related to the requirements and the amount of written documentation.



- 1. Introduction**
 - 1.1 Purpose
 - 1.2 Document conventions
 - 1.3 Project scope
 - 1.4 References
- 2. Overall description**
 - 2.1 Product perspective
 - 2.2 User classes and characteristics
 - 2.3 Operating environment
 - 2.4 Design and implementation constraints
 - 2.5 Assumptions and dependencies
- 3. System features**
 - 3.x System feature X
 - 3.x.1 Description
 - 3.x.2 Functional requirements
- 4. Data requirements**
 - 4.1 Logical data model
 - 4.2 Data dictionary
 - 4.3 Reports
 - 4.4 Data acquisition, integrity, retention, and disposal
- 5. External interface requirements**
 - 5.1 User interfaces
 - 5.2 Software interfaces
 - 5.3 Hardware interfaces
 - 5.4 Communications interfaces
- 6. Quality attributes**
 - 6.1 Usability
 - 6.2 Performance
 - 6.3 Security
 - 6.4 Safety
 - 6.x [others]
- 7. Internationalization and localization requirements**
- 8. Other requirements**
- Appendix A: Glossary**
- Appendix B: Analysis models**

Figure 3 SRS template [Adapted from Wiegers & Beatty (2013, p. 12)]

1.4.1 Waterfall approach

The predictive approach of the waterfall model, attempting to minimize project risk through extensive planning and documentation, assumes a fully assembled SRS document before software development begins (Wiegers, 2013). As described and illustrated by Robertson and Robertson in their book (2012, p. 24), the waterfall model is a standard development model in which all development phases are implemented sequentially - the next phase begins only after the previous phase is fully completed (see Figure 4). It follows that in this type of approach, the requirements should be clear and easy to understand so that there are no critical mistakes in the subsequent phases. To write an SRS document in the waterfall model, it is recommended to start with an outline that should include important points such as the purpose, project scope, user needs, system features, and requirements (see Figure 3).

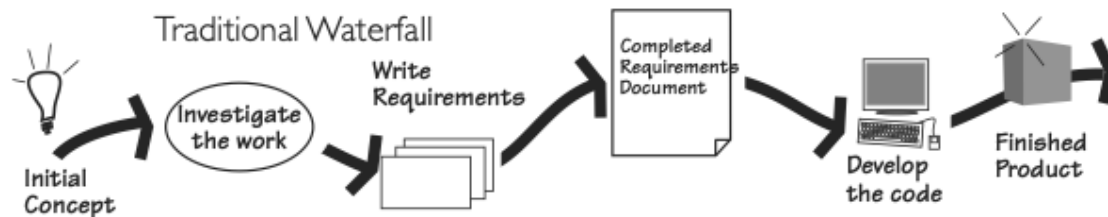


Figure 4 SRS document preparation by the waterfall approach [Adapted from Robertson & Robertson (2012, str. 24)]

1.4.2 Agile approach

Agile methodologies focus on iterative and incremental development and divide software development into short cycles called iterations. As mentioned by authors Roudenský and Havlíčková, agile methodology also includes such well-known and popular frameworks as Scrum and Extreme Programming (2013). During each iteration, the development team adds a small set of features based on the priorities set by the customer, tests them to ensure that the software works as intended, and validates them using acceptance criteria set by the customer (Wiegers & Beatty, 2013). This makes the process of creating an SRS document more flexible. In this approach, the business analyst iteratively (usually every second or fourth week) collects feedback from the customer and makes adjustments to the SRS document or user stories (see Figure 5).

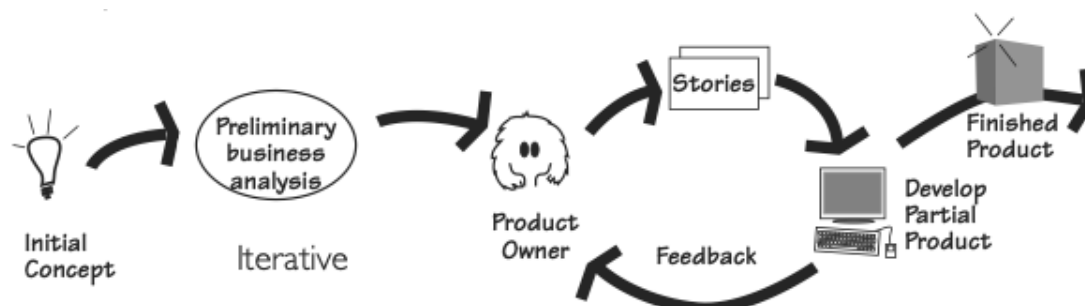


Figure 5 Preparation of user stories by the iterative approach [Adapted from Robertson & Robertson (2012, str. 24)]

User stories are mostly used specifically in agile projects and describe the functionality of the system from the perspective of users. There is a specific format that is used for writing conditions in user stories. This format is adapted by each team depending on the project. For example:

As <User/Role/Persona>

I want <Functionality>

To get <Business value>

1.5 Characteristics of good requirements

Undoubtedly, writing an effective software requirements documentation to achieve a common understanding of what the system should do must be accompanied by some guidelines. The more precise the SRS document is written and the more details the developers have before the development process begins, the more effective the work will be. Of course, each company approaches software requirements writing differently, depending on what helps its teams better understand the requirements in order to develop a proper product. Based on various sources (Robertson & Robertson, 2012; Wiegers & Beatty, 2013; Jama Software, 2022), there are some fundamental practices that can be used to effectively document software requirements. These practices can be divided into 5 common categories.

Clarity

When formulating requirements, it is important to use simple words and avoid general phrases. Simple sentences that reflect the purpose of the specific functionality are clear and easy for everyone to understand (Wiegers & Beatty, 2013). As a result, less time is spent on the unnecessary communication and specifying the details, which leads to the higher effectivity of team members and finishing work on time, meeting the deadline.

Unambiguity

Unambiguity of requirements creates a clear distinction between what the system is supposed to do and what it eventually does after it is implemented. If the formulation of the requirements is interpreted differently by developers, users, and other project stakeholders, it may well be that the system is completely different from what the user imagined at the beginning (Robertson & Robertson, 2012).

Completion

The requirements set is complete when it describes all the essential information that the developer needs, including requirements related to functionality, performance, design constraints and system responses. As mentioned in the Jama Software (2022, n.p.) article, *“Using minimal requirements specifications runs the risk of having different people fill in the blanks in different ways, based on different assumptions and decisions.”* It follows that, specifying all the needed information before the construction begins is important to make sure that the requirements are complete.

Prioritization

Developers, clients, and other stakeholders should rank the individual requirements according to their importance to the client and their stability. If resources are insufficient to implement all requirements within the allocated time and budget, it is very useful to know which requirements are not so mandatory and which are considered as critical (Jama Software, 2022).

Traceability

Requirements must be traceable, in other words, it means that each of the requirements should have a unique, persistent number or identifier and be written in a structured way (Robertson & Robertson, 2012). Developers can use traceability of requirements both to gain a better understanding of the design and to provide a higher level of assurance that all requirements are met by a given implementation.

2 Test analysis methods

Choosing and applying the right test design technique when developing test cases eventually guarantees a high-quality testing of the software. This chapter focuses on **test analysis methods and covers the main terms as well as some general guidelines for designing test cases**. Subsection 2.1 introduces the terms related to the test case design and summarizes the best practices of writing good quality test cases. Subsection 2.2 explains and outlines specification-based test case design techniques. The last subsection 2.3 is dedicated to the testing techniques based on the experience.

2.1 Test case design

According to the International Software Testing Qualifications Board (ISTQB) [2021, p. 23] Glossary of Terms, a test design is “*the activity of deriving and specifying test cases from test conditions.*” Before exploring guidelines for writing test cases more precisely and defining test design techniques, it is important to introduce such key terms as test idea and test case.

2.1.1 Test idea

The test idea responds to the question: "What can be tested?". Roudenský and Havlíčková (2013) characterize a test idea as a short description of what might be appropriate to test. Test ideas serve as an initial source of information for test cases and indicate what components of the system must then be verified using test cases. The list of test ideas consists of a simple document that includes the problem areas of the developed software and the behaviour of the developed IS/ICT described in the form of "action and reaction" (Rejnková, n.d.).

2.1.2 Test case

As a result of thinking about what parts of the system can be tested and developing test ideas, the process of designing test cases can be started. In compliance with the ISTQB Glossary of Terms (2021, p. 22), a test case is "a *set of preconditions, inputs, expected results, output conditions, and actions that are developed based on the test conditions.*" In other words, a test case is a set of actions/steps performed to verify a particular feature or function of a software application. Although there is no formal way of documenting test cases, there is a standard template with test case fields that is commonly used:

- Test Case ID
- Test Description
- Test Data
- Steps to be executed
- Expected result

- Actual result
- Pass/Fail

2.1.3 Test case writing guidelines

Well-written test cases help testers to verify the software, by having a detailed instruction of how to test it. Looking at the quality of software from this perspective, it can be stated that the accuracy of written test cases is one of the main factors in the future success of the system. A good, structured test case can guide a tester through the system and help him to find more bugs. Therefore, it is important to know how to write test cases efficiently and effectively. The following guidelines are based on the test case format that is used in this thesis and described in Subsection 2.1.2.

By analyzing the typical structure of a test case, it is possible to highlight certain required attributes that influence the correct design of test cases. For example, each test case should have a simple "short" name and a unique ID that represents it. This helps in the organization and use of test cases in subsequent phases (IEEE Computer Society, 2005). The test case description should detail what functions or features this test case is supposed to verify. Identifying all the information and data that are needed to execute the test case, also helps to make the job of a tester faster and easier during test execution.

Since test cases include "step-by-step" instruction, steps described in a test case play the main part of it. To avoid a misunderstanding between team members and to ensure that the system is tested accordingly, test steps must be clear, understandable, and consistent (Bartlett, 2022).

Another important attribute of test cases is an expected result. The expected result tells testers what they should expect after performing each of the test steps, while the actual result reflects the current state of the system under test (Black et al., 2017). To ensure the correct result of a test performed, the expected results should be stated simply and clearly so that a comparison between the actual and expected results can be easily made.

In summary, the following fundamental guidelines for writing good test cases can be highlighted:

- Each test case should be identified with a unique ID and name characterizing the module under test.
- The test case must be described in a clear manner, without using special terminology or complex language, but without missing important information.
- Test steps must be written in a plain and brief form so that anyone can perform them, as well as the number of steps should be optimal (3-8).
- Steps should be also complemented with easy-to-understand expected results; this is essential to facilitate error handling.

2.2 Specification-based techniques

Test cases can be derived directly from the specifications and designed accordingly to the applied methods. Specification-based testing that is also known as black-box testing is a testing method in which tester only focuses on the outer layer of software, without looking at the internal code design or the structure of software. Hambling et al. (2010, p. 82) points out that: “*Black-box techniques are based on an analysis of the test basis documentation, including both functional and non-functional aspects.*” This chapter is dedicated to introducing some of the black-box techniques that help in reducing the risk of missing an error and improving the quality of the test results.

2.2.1 Equivalence partitioning

This method is based on a simple idea: input data are divided into equivalence partitions, according to the principle that the program behaves the same way with each representative of a separate partition. The input values are split into at least two partitions: the first one contains a list of valid input values, and the second one contains a list of invalid input values (Hambling et al., 2010). This way it is not necessary to test all possible inputs, only one representative of the partition needs to be tested. For instance, the online library application offers students a discount by validating an age input field. An age between 18 and 26 is considered valid, while an age less than or equal to 17 and greater than or equal to 27 is invalid. The equivalence partitions then look like this:

Equivalence partitioning		
Invalid	Valid	Invalid
≤ 17	18-26	≥ 27

Table 1 Example of the equivalence partitioning [Source: own elaboration]

2.2.2 Boundary value analysis

Boundary value analysis is another test case design technique that can also be combined with the equivalence partitioning method. This technique involves defining boundary values and evaluates whether this certain range of values is acceptable for the system or not. Tests can be created to cover both invalid and valid boundary values (Rajkumar, 2018). Many systems tend to behave incorrectly at boundary values, so evaluation of application boundary values is very important. As in the previous technique, it is required to define equivalence partitions before finding boundary values. This step is essential, because the effectiveness of the boundary value tests depends on how well the equivalence classes are partitioned. The example of the age field (18-26) that was previously applied for equivalence partitioning, can also be used for finding boundary values see Table 2.

Boundary value analysis		
Invalid (min -1)	Valid (min, min+, -max, max)	Invalid (max +1)
17	18, 19, 25, 26	27

Table 2 Example of the boundary value analysis [Source: own elaboration]

2.2.3 Decision table testing

Decision table testing is one of the effective methods of black-box testing. If the system behavior is different in each set of input data, creating a decision table is a perfect solution to see all possible combinations of conditions and actions they produce. In other words, a decision table is used to test different combinations of inputs that result in different outputs within the program (Hamilton, 2022). This is a very systematic approach to testing when testers record the input combinations and their results in a table format. In decision tables, the conditions, including the output, are written horizontally, while the rules (a particular combination of input conditions) are shown vertically. Consequently, the number of rules depends on the number of conditions and is calculated by the formula 2^n , where n is the number of conditions (Spillner et al., 2014). Different notations are used to illustrate the states of the conditions: T (True)/F (False) or analogous 0/1 values. Table 3 shows a table decision for a login page: the user needs to provide his correct email and password to get redirected to the profile page.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username	F	T	F	T
Password	F	F	T	T
Output	F	F	F	T

Table 3 Example of the decision table testing [Adapted from Hamilton (2022)]

2.2.4 State transition testing

State transition testing helps to analyze the behavior of an application that is triggered by the transitions between current and past states of the application (Hambling et al., 2010). If a tester needs to control different system transitions, this black-box technique would be the most appropriate to use. The diagram that is used to illustrate such transitions is called a state transition diagram, in which circles represent states and arrows represent transitions between states. Figure 13 demonstrates a diagram with the specific example of state transitions within the application: In order to activate the profile of a new client, the application must first validate his ID card. Once the ID card has been validated, the profile becomes active. If the ID card does not pass validation, the client's profile stays non-active.

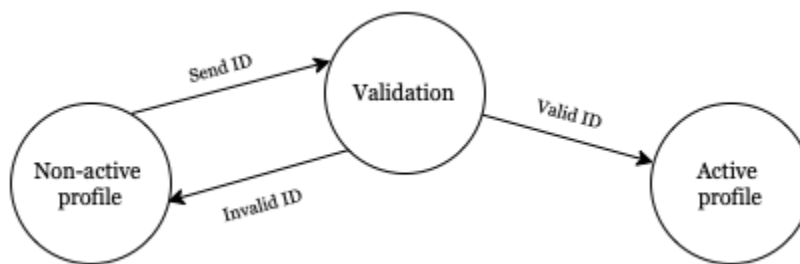


Figure 6 Example of the state transition diagram [Source: own elaboration]

2.3 Experience-based techniques

An approach in testing that involves simultaneously exploring the software, designing test cases in mind, and immediately executing them without writing any test documentation is called an experience-based testing. This type of testing is not predefined in the test plan and such tests are created, executed, and modified dynamically as needed. Experience-based testing allows testers to use their intuition, imagination, and their personal knowledge to help in finding more defects in the system, using specific techniques (Black et al., 2017).

2.3.1 Exploratory testing

The ISQTB Glossary of Terms (2021, p.8), defines exploratory testing as “*An informal test design technique where the tester actively controls the design of the tests as those tests are performed. The tester uses the information gained while testing to design new and better tests.*” This technique offers to its user two advantages. The first advantage is that this method makes it possible to test software without having specifications or well-written requirements. Another advantage which mentions Spillner et al. (2014, p.176) is about the timing: “*The technique is also applicable when time is severely restricted because it uses much less time than other techniques.*” In other words, exploratory testing allows to run tests on a system in a limited range of time when the requirements documents are not available, primarily depending on the tester skills.

2.3.2 Ad hoc testing

Ad hoc testing is a type of informal testing that is performed without test preparation, without defining expected results, or designing test cases. Compared to the exploratory testing, which is more structured, ad hoc technique is more intuitive and random, meaning that the tester just tests what he wants in the system. The authors Roudenský and Havlíčková (2013) state that ad hoc testing is typically an activity conducted unsystematically, without planning and randomly, with the aim of discovering as many defects as possible. This allows testers to step out of a systematic approach and find errors that could not have been discovered if they had followed precise written test cases.

2.3.3 Error guessing

As stated by Spillner et al. (2014, p.255) this method is *“a test design technique where the experience of the tester is used to anticipate what defects might be present in the component or system under test as a result of errors made and to design tests specifically to expose them.”* In error guessing the intuitive and deductive abilities of the tester are brought into play to detect errors in advance, expose them, and then report them. As well as two other techniques error guessing is based primarily on the experience of the tester, his knowledge of the product and developed skills of identifying weak points of the system. The test cases are based on where faults have occurred in the past or the tester's ideas of where faults might occur in the future.

3 Test management tools

This chapter deals with a **definition of a test management tool, its advantages and describes the most commonly used tool TestRail**. Undoubtedly, test cases can be written, organized, and kept in different tabular forms using various platforms and templates. However, the growing number of test cases and the need of their regular modifications dependent on the changes of the system makes it difficult to keep track of test cases in such format. For this particular reason, the test management tools that help managing different activities related to testing became increasingly popular.

Today's market offers companies the opportunity to make their testing process simpler and more manageable by choosing a suitable test tool. Even though specific features of test tools can vary from one to another, they are all used for improving efficiency of testing process. The ISTQB Glossary of Terms (2022, p. 26) defines a test tool as: *“A software product that supports one or more test activities, such as planning and control, specification, building initial files and data, test execution and test analysis.”*

In addition to the different types of testing tools, they can also be categorized by the testing activities they support and enhance. This work explicitly focuses on test management tools that help manage the testing process throughout the development lifecycle. According to the Spillner et al. (2014, p.169): *“Test management tools provide mechanisms for easy documentation, prioritization, listing, and maintenance of test cases. They allow the tester to document and evaluate if, when, and how often a test case has been executed.”*

Nowadays organizations have a wide range of test management tools. Although each of them has its own distinctive features, there are some common advantages. They are based on the list of individual test management tools defined by Hamilton (2020):

- Test management tools help to organize, manage, and track all the testing activities at one place.
- They enable to monitor and create necessary metrics related to testing process (reports, diagrams, tables).
- In addition, test management tools provide easy-to-use interface for designing test cases, executing them, and scheduling test plans.
- They are also used to support effective cooperation between team members and can be adapted to each project individually.

3.1 TestRail

TestRail was chosen in this thesis as the most well-known test management solutions by this date, to show the practical use of a test management tool with the test cases designed in the following chapters. Since TestRail was released by Gurock Software in 2010, it has become the number 1 modern test management tool, receiving the highest ratings from more than 100,000 development and QA team members (TestRail, 2020). Official website

of TestRail (2020) states that, TestRail is a web-based test management tool used by testers, developers, and other team members participating in software development to organize, manage, and track software testing activities. Main characteristics of TestRail include:

Methodology support - TestRail can easily be used in both methodologies: Agile and Waterfall.

Test support - it can support different types of testing: Black-box testing, Exploratory testing, Functional testing, and Test Automation.

Test case management – user-friendly interface for designing and managing test cases, suites and plans efficiently.

Integration with bug trackers, automation tools and other add-ons, including Jira, Bugzilla, Selenium, GitHub, and many more.

Developed reporting system – reporting section of TestRail makes it easy to generate project reports, collect necessary statics, share test results, and create other necessary metrics.

Management of multiple projects – allows team member to create comfortable and flexible project organization for the best cooperation.

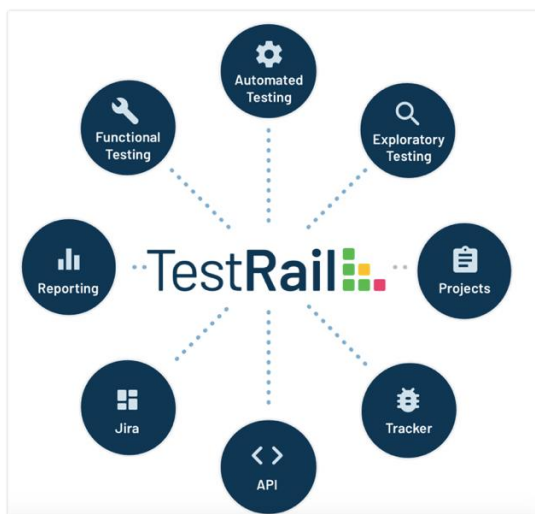


Figure 7 Features of TestRail [Source: TestRail, 2020]

The tabs of TestRail including their descriptions which are presented in the following sections are used in the practical part of this thesis:

Dashboards

After creating and account and logging in to TestRail, the first page that is shown is the **main dashboard**. This page displays all projects and recent activities.

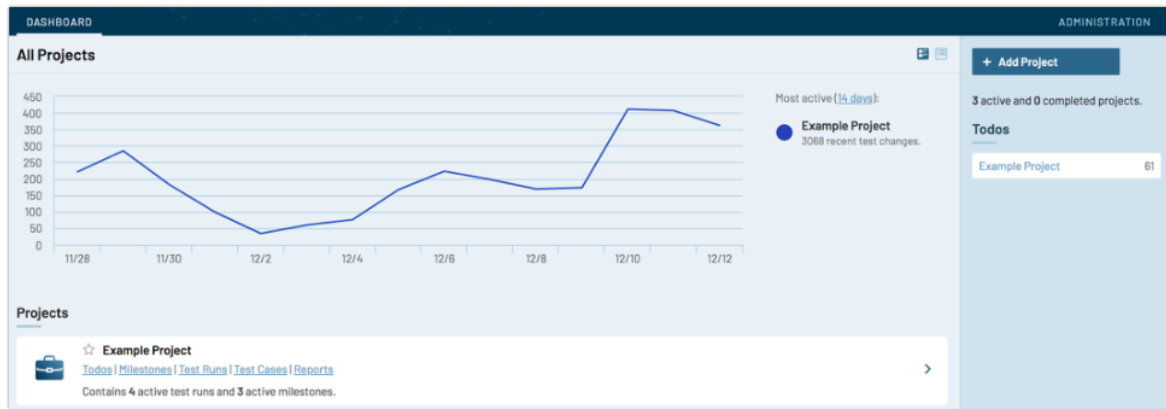


Figure 8 Main dashboard [Source: TestRail, 2020]

Clicking on a project title loads the **project dashboard** which provides an overview of the project, including a snapshot, a list of recently completed tests, test runs, reports, and upcoming milestones.

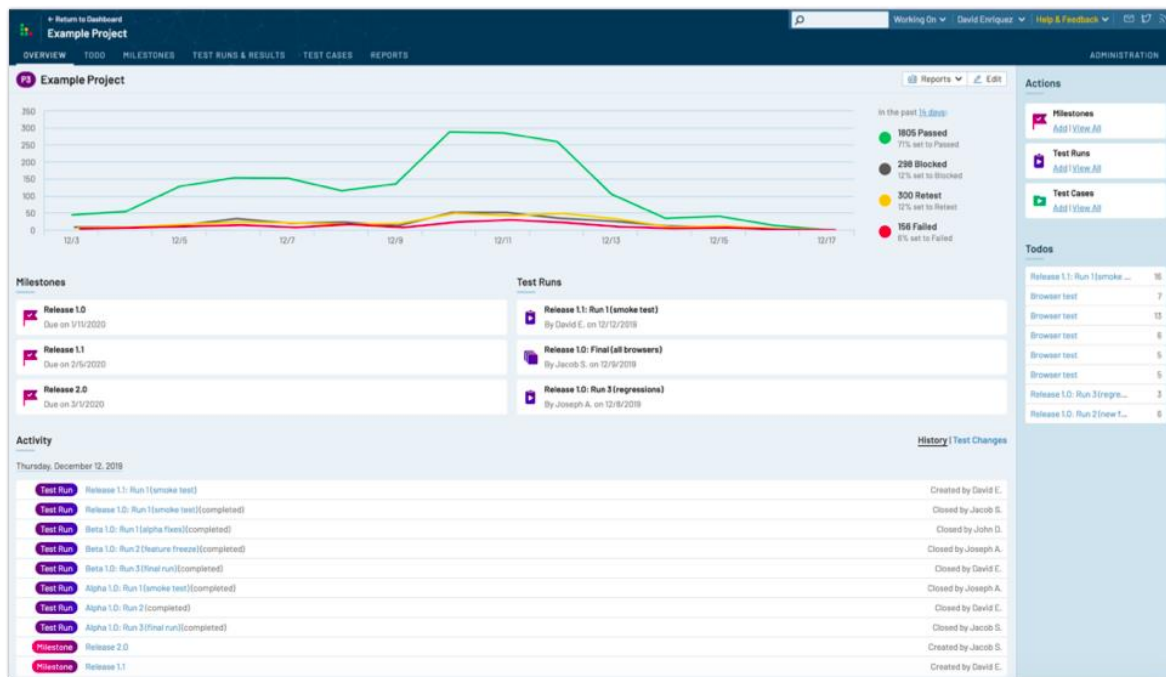


Figure 9 Project dashboard [Source: TestRail, 2020]

The project dashboard allows users to switch to the other sections, as shown in the figure below:



Figure 10 Main sections [Source: TestRail, 2020]

Considering that one of the goals of this thesis is to apply selected written test cases to the TestRail, it was decided to describe next two sections in more details: Test Runs & Results and Test Cases.

Test Cases

Test cases in TestRail can be divided into sections and sub-sections, which allows testers to group test cases by specific features and functionalities. To add a new sub-section, users need to click on the “**Add Subsection**” button, and after creating it, go to its page, where they can already create necessary test cases using the “**Add Case**”.

Each template of a test case in TestRail consists of a **Title**, **Preconditions**, **Steps** and **Expected result** (see 2.1.2). Besides that, it has other required attributes such as: Section, Template (exploratory session, steps, text), Type and Priority. In addition to that, TestRail also allows to expand test cases with graphical information and tables. Completed test cases can be transferred between sections, copied, exported to XML, Excel/CSV, or imported from an XML or CSV file into TestRail.

The screenshot shows the 'Test Case Example' form in TestRail. The form is titled 'C41 Test Case Example'. It includes the following fields and sections:

- Title ***: A text input field containing 'Test Case Example'.
- Section ***: A dropdown menu.
- Template ***: A dropdown menu containing 'Test Case (Text)'.
- Type ***: A dropdown menu containing 'Other'.
- Priority ***: A dropdown menu containing 'Medium'.
- Estimate**: A text input field.
- References**: A text input field.
- Automation Type**: A dropdown menu containing 'None'.
- Preconditions**: A section with a text area for preconditions. Below the text area, it says: 'The preconditions of this test case. Reference other test cases with [C#](e.g. [C17]).'
- Steps**: A section with a text area for steps. Below the text area, it says: 'The required steps to execute the test case.'
- Expected Result**: A section with a text area for the expected result.

Figure 11 Test Case Template [Source: TestRail, 2020]

Test Runs & Results

The users of TestRail can create test runs in this section by clicking on the “**Add Test Run**” button and follow the results in the form of report. First, it is necessary to specify the name for the planned test run and to add additional information as needed (Milestone, Assign To, Description). After that it is important to choose test cases that will be included in the test run: **all test cases**, **specific test cases** or **test cases based on filter selection**.

After creating a test run, the tester can proceed with its direct execution. He must specify the status of each test case, the name of the executor and monitor the current results and

statistics, as shown in Figure 11. By default, each test case has five different statuses that are shown in Figure 12. Status of a test case can be chosen directly on test page by clicking on the “**Add Result**” button or changed on the run page.

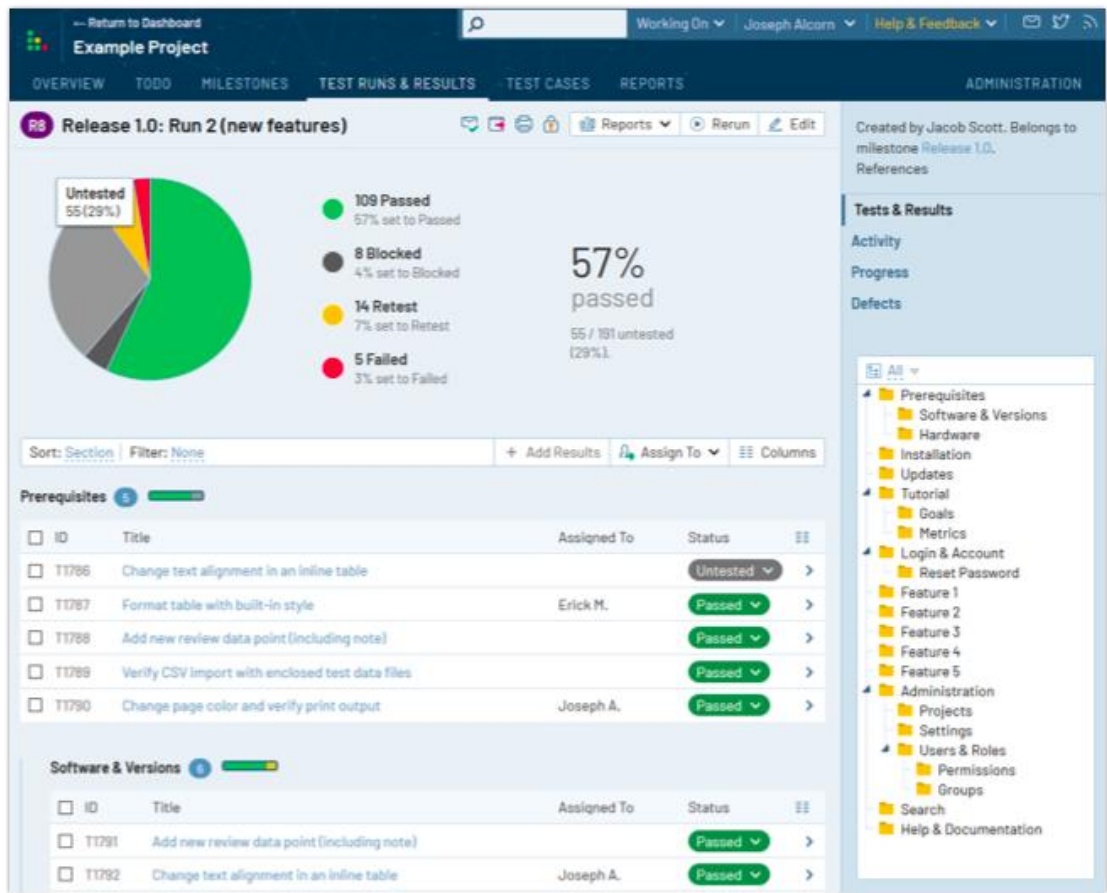


Figure 12 Test Run [Source: TestRail, 2020]

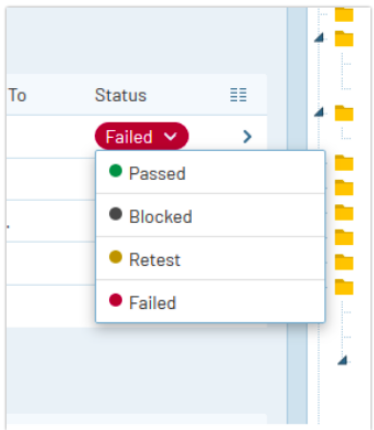


Figure 13 Test statuses [Source: TestRail, 2020]

4 Requirements analysis of the tested application

This chapter contains detailed **information about the functionality of the Pilulka mobile application and deals with selected functional requirements** chosen for the design of test cases in the following chapters. Subsection 4.1 focuses on the domain of the chosen mobile application in the online pharmacy market. Subsection 4.2 then presents individual screens of the application with a detailed functionality. Subsection 4.3 of this chapter covers selected functional requirements in the form of user stories. The last subsection contains a traceability matrix.

4.1 Specification of application domain

The online pharmacy sector is now considered the fastest-growing segment of e-commerce. According to Fittler et al. (2018, p. 1), an online pharmacy is “***an internet-based vendor (legal or illegal), which sells medicine and may operate as an independent internet-only site, an online branch of “brick-and-mortar” pharmacy.***” With the outbreak of a global pandemic of coronavirus disease in 2019 (COVID -19), people have had to radically change their lifestyles in a very short time. As the world's population had to quickly adapt to the changes associated with the pandemic restrictions and lockdowns, people became extremely demanding when it came to online solutions for their daily needs. While the pandemic posed major challenges to numerous industries around the world, the online pharmacy market, in contrast, has become a technological innovation (Facts & Factors, 2022).

Today online pharmacies continue to be in demand by offering their customers a wide selection of medications, attractive product prices, and contactless home delivery options. The successful adaptation of digital solutions for the pharmaceutical industry has significantly changed the global pharmacy market. According to AllTheResearch's (2021, p.1) market research study, “***The Global Online Pharmacy market was valued at USD 68.2 billion in 2020, and it is expected to reach USD 202.3 billion by 2027, with a compound annual growth rate (CAGR) of 16.8%, during the forecast period from 2021 to 2027.***”

4.2 Description of the application Pilulka

Pilulka Lékárny a.s. is a technology company operating on the markets of the Czech Republic, Slovakia and Romania since 2013. In addition to running more than 150 physical pharmacies, Pilulka.cz is the largest mail-order pharmacy in the Czech Republic. It offers its customers a wide range of high-quality medicines, healthcare services and health products such as food supplements, vitamins, sports nutrition, cosmetics, personal care

products and other categories (Pilulka.cz, 2021). Pilulka.cz not only provides a convenient online pharmacy solution with a fast delivery service but also actively motivates its users to lead a healthy lifestyle by involving them in the unique loyalty program called Pilulka Fit.

The mobile application Pilulka was launched in 2017 and is now the most successful pharmacy application on the Czech market (Pilulka.cz, 2021). In addition to the Czech Republic, the application is currently available in two other countries: Romania and Slovakia, with support for each of these languages. The application can be downloaded from the Play Market for the Android platform and the App Store for IOS. However, this thesis deals only with the IOS version of the application and its latest available version, at the time of writing this thesis, which is 6.23. Apart from the ordering process, it is necessary to detail other main features of the Pilulka mobile application:

- Tracking of Pilulka Cars on the map
- ePrescription reservations
- EAN scanner
- Repeated purchases
- Pilulka Fit - rewards with credits for an active lifestyle

Next subsections present individual screens of the application and describe their main functions. Since the version of the application, considered in this work, is in Czech, the original Czech names of the screens are given in addition to the English translations.

4.2.1 Profile

The profile screen “**Profil**” offers the option of creating an account or logging in via email, Google Account, Apple ID, or Facebook. Once the user has logged in, the profile screen provides access to features such as viewing the history of purchases already made, browsing favourite products, viewing the number of credits, editing personal information, and setting up preferences.

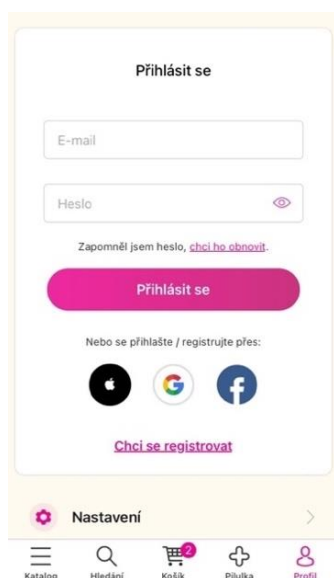


Figure 15 Log in screen

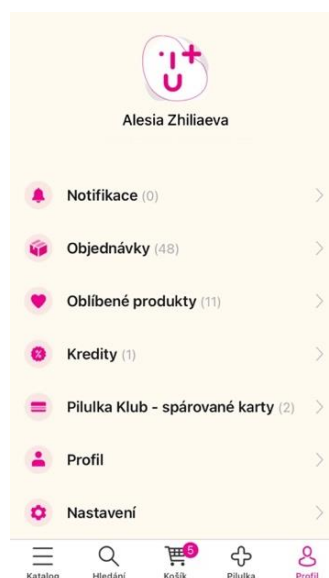


Figure 14 Profile screen

4.2.2 Settings

Through the settings “**Nastavení**”, the user can choose if he wants to receive notifications and exclusive offers. Here it is also possible to switch the application to dark or light mode and to change the country of the application.

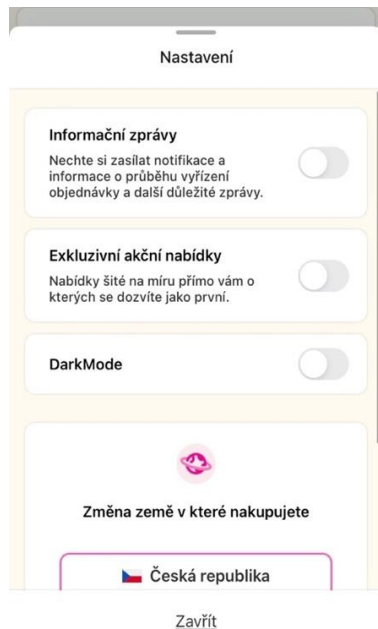


Figure 16 Settings

4.2.3 Shopping cart

The shopping cart “**Košík**” is the part of the application where customers can place their orders. This screen displays all the items that the user places in his shopping basket. The order can be completed by selecting delivery, payment method and entering personal data such as mobile phone and address. The user can also use his earned credits or voucher and add a note if needed.

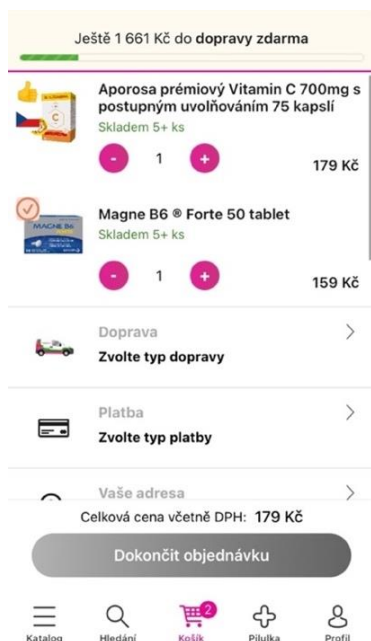


Figure 17 Shopping cart

4.2.4 Search

The search screen “**Hledání**” allows users to search for products, their categories or brands via the search box and displays the search history. Instead of finding products by typing it is also possible to scan the EAN of the product using the camera.

4.2.5 Catalogue

This screen “**Katalog**” is the homepage of the application. It shows relevant seasonal banners, the main products categories from which people can view individual products and various offer carousels such as favourite products, personal recommendations, sales and offers of the day. From this screen users can also navigate to the Pilulka Box, Pilulka Auto and ePrescription sections. In addition, homepage lets a user join the Pilulka Fit challenges if he is not connected yet and displays the results if he already is the participant. If the user completed his order, homepage provides notifications about the status of his order at the top of the screen.

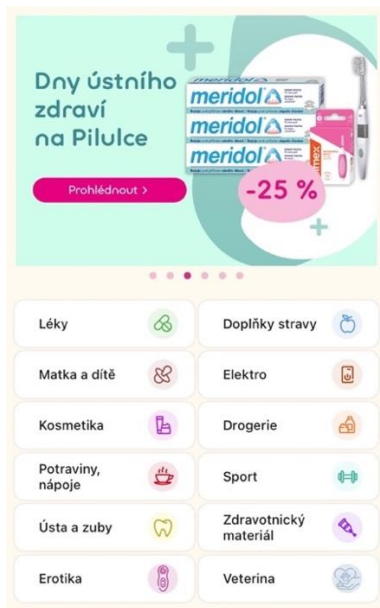


Figure 19 Catalogue

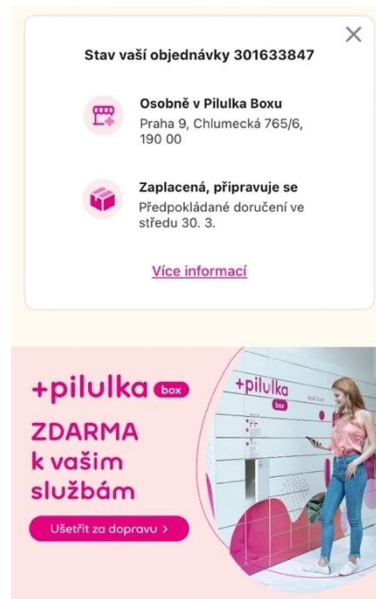


Figure 18 Notifications

4.2.6 Pilulka Fit

The user only has a full access to this section if he has an account to which he is logged in. He then has the option to select, join and leave various walking, running, cycling and challenges. The centre of the screen always shows the number of steps the user has taken per day, which can also be displayed in kilometres. The statistics of the user's movement can also be viewed in months and weeks. After joining a challenge and opening the details of it, the user can see his position among the other participants, the scale of his results and credits earned accordingly.

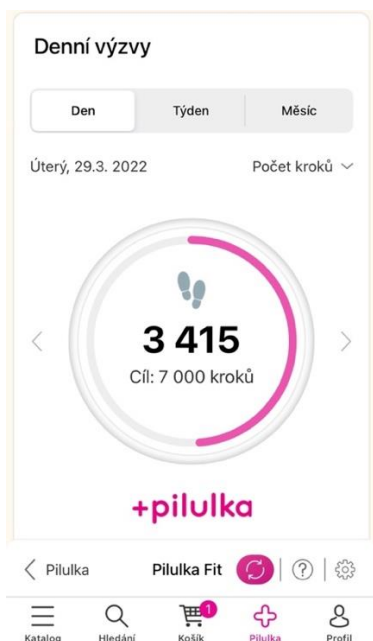


Figure 21 Step counter

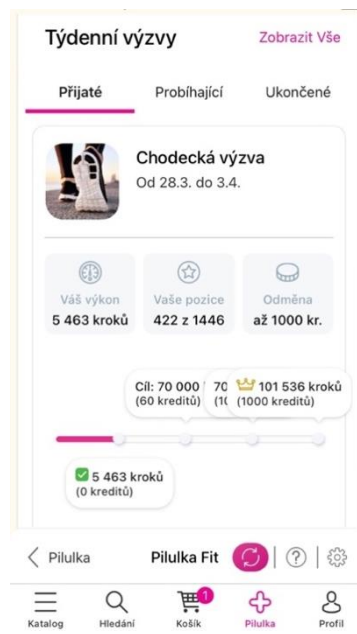


Figure 20 Weekly challenges

4.3 Selected functional requirements

Pilulka's mobile development team uses an agile approach implemented in the framework known as Scrum (see 1.4.2). Developers work on their tasks in short sprints that last in total 14 days, constantly improving the quality of the application. The requirements implementation process starts with a business owner who first specifies the requirement and determines together with the product owner the priority of this requirement. After that product owner prepares sprint backlog which consists of fixed list of user stories (see 1.4.2) for a given sprint. Before the start of each sprint, the team decides together with the product owner which tasks should be solved in the next sprint, based on the velocity, priorities, and capacity of the team members. The mobile team in Pilulka also has regular meetings, called groomings, where developers, testers and a product owner go through the user stories, estimate them, discuss certain requirements and decide how best to solve and test them.

Table 4 contains selected functional requirements for the Pilulka mobile application that serve as the basis for developing test cases in the following chapters. Some of the functional requirements covered in this thesis were received from the mobile development team, others were created by the author based on her personal knowledge of the application to describe the features that author found important to include. These requirements are recorded in the form of user stories, where each requirement has its own unique identification number and a title characterizing main functionality. Requirements named **Free Delivery**, **Product review** and **Admission to the Pilulka Fit club** were selected by the author to implement specification-based design techniques presented in Subsection 2.2. Other requirements that focus on interesting features from different parts of the application are used to develop test cases based on user stories.

Table 4 List of selected functional requirements [Source: own elaboration]

ID	Title	User Story	Acceptance Criteria
US-1	Free delivery	As a user, I want to be able to have a free delivery service when I have a bigger amount of products/higher price products in my cart so that I can enjoy online shopping and have some benefits with Pilulka.	• Provide an option to count how much is left for a free delivery at the top of the screen, depending on the total purchase amount in a shopping cart. • Free delivery for all types of delivery is available from 1999 Kč. • Show Free Delivery message “<i>Dopravu máte zdarma</i>” once the quantity has reached 1999 Kč.

US-2	Product review	As a user, I want to give my feedback on the products in the application so that I can share my purchase experience with others and show whether I was satisfied or not as a customer.	• Show button „<i>Hodnotit produkt</i>“ in the individual section <i>Hodnocení</i>. • Display a form with required fields to leave a review.
US-3	Admission to the Pilulka Fit club	As a user, I want to join Pilulka Fit club, so that I can have a healthier lifestyle by using Pilulka application.	• Both homepage and Pilulka services page will link to the Pilulka Fit club. • Provide an option to accept all registered and logged in users into Pilulka Fit club. • Display login page if the user is not logged in. • Ask a user after he logs in if he wants to give access to the Health application with his data.
US-4	Login	As a non-logged in user, I want to be able to log in to the application via email, Google Account, Facebook, or Apple ID so that I can access other features of the application.	• Display a window for log in • Provide an option to choose between Google Account, Facebook, and Apple ID. • Only valid login and password are accepted • Display error messages if the email or password is invalid. • Display user's profile after successful log in.
US-5	Favourite products	As a logged in user, I want to mark products that I like and save them in the separate list named „ <i>Oblíbené produkty</i> “ so that it is more convenient for me to have my favourite products in one place and reorder them.	• Show the heart mark on each product. • Differentiate colour of the unmarked/marked heart. • Display „<i>Oblíbené produkty</i>“ section in the profile of the user. • Provide an option to add and remove favourite products from the list.
US-6	Repeat order	As a logged in user, I want to repeat my previously placed orders, so that I do not have to spend time on searching for products I already once ordered or order regularly.	• Provide an option to repeat orders in the section „<i>Objednávky</i>“. • Display all the items from the repeated order in a shopping cart after.
US-7	EAN scanner	As a user, I want to search a product by scanning the EAN code, so that the process of	• Provide an option to scan a EAN code through

		finding products for me is faster and easier.	camera at the search screen. • Display a window through which user can scan the product. • Add flashlight option. • Show correct results accordingly to the EAN code.
US-8	Substitution of products	As a user, I want to have opportunity to replace products at my cart with the status non-available “ <i>Není dostupné</i> ” or out-of-stock “ <i>Skladem u dodavatele</i> ” by their substitutes that are available and in stock right now, so that I can order a similar product instead of the one that is no longer available/have long delivery time and stay satisfied with Pilulka.	• Display non-available products in a shopping cart as grey with a red frame around. • Display out-of-stock products in orange frame. • Show as substitutes only products that are in stock by clicking on the button “<i>Nahradiť</i>”. • Provide the option to delete this product from a cart with a button “<i>Odstrániť</i>”.
US-9	Country of the application	As a user, I want to be able to change countries of the application, so that I can use Pilulka internationally.	• Provide an option in settings to switch between 3 countries: CZ, SK, RO. • The country of the application should be set automatically after installation, based on the phone’s region settings. • If the region of the user is not set on CZ, SK, RO application offers to manually choose the country after the installation.

4.4 Traceability matrix

The Requirements Traceability Matrix (RTM) is used to link requirements to test cases. This document helps in easier tracking of individual test cases and in identifying whether each of the requirements has been covered by a test. Table 5 maps the IDs and titles of the user stories presented in the previous subsection to the IDs of the test cases.

Table 5 Traceability matrix

User Story ID	User Story Title	Test Case ID
US-1	Free delivery	TC_01.01
US-2	Product review	TC_02.01
		TC_02.02
		TC_02.03
		TC_02.04
		TC_02.05
		TC_02.06
US-3	Admission to the Pilulka Fit club	TC_03.01
		TC_03.02
		TC_03.03
		TC_03.04
US-4	Login	TC_04.01
		TC_04.02
US-5	Favourite products	TC_05.01
		TC_05.02
US-6	Repeat order	TC_06.01
		TC_06.02
US-7	EAN scanner	TC_07.01
US-8	Substitution of products	TC_08.01
		TC_08.02
US-9	Country of the application	TC_09.01
		TC_09.02

5 Application of specification-based testing techniques

The aim of this chapter is **to apply the specification-based testing techniques** presented in the Subsection 2.2 **to selected requirements** described in Table 4 (see 4.3). It was decided to use 3 user stories for 3 different techniques: **US-1 Free delivery**, **US-2 Product review** and **US-3 Admission to the Pilulka Fit club**. Test cases developed using the techniques employed in this chapter are recorded in the same format using a template presented in Chapter 6 and can be found in the annexes of this work (see annex A, B, C).

5.1 Free delivery

The test cases for the **US-1** named Free delivery are designed by combining boundary value analysis (see 2.2.2) and equivalence partitioning (see 2.2.1). Clearly, to test the functionality of the Free delivery option, it is impossible to check all the different combinations of purchase amounts. For this reason, this subsection uses equivalence partitioning to divide the input values into two partitions: valid and invalid. Furthermore, boundary value analysis obtains the boundary value from each of these partitions, as shown in Table 6. To clarify how this function works, it is important to mention that free delivery is activated when the purchase amount in Kč reaches 1999. This can be recognized by the info message at the top of the shopping cart screen "*Dopravu máte zdarma*" and the option to choose all the delivery methods for free. If the purchase amount is below 1999 Kč, free delivery is not activated.

Table 6 Equivalence partitioning and a boundary value analysis – Free Delivery [Source: own elaboration]

Equivalence partitions	Boundary value analysis	Delivery
<1999	1998	Scale indicating how much is left before free delivery
=>1999	1999, 2000	Free delivery

5.2 Product review

To leave a review about a particular product, in addition to mandatory star rating the user should select one of the 3 required attributes to fill in: positive point, negative point, or text.

Table 7, based on the decision table method (see 2.2.3), demonstrates all possible combinations of conditions that allow or do not allow the user to submit a review by clicking on the "Odeslat hodnocení" button. If the conditions are not met, the "Odeslat hodnocení" button remains grey. If at least one of the required attributes is filled in and a star rating has been made, the button turns pink, which means that the user can submit his review.

Table 7 The decision table - Product review [Source: own elaboration]

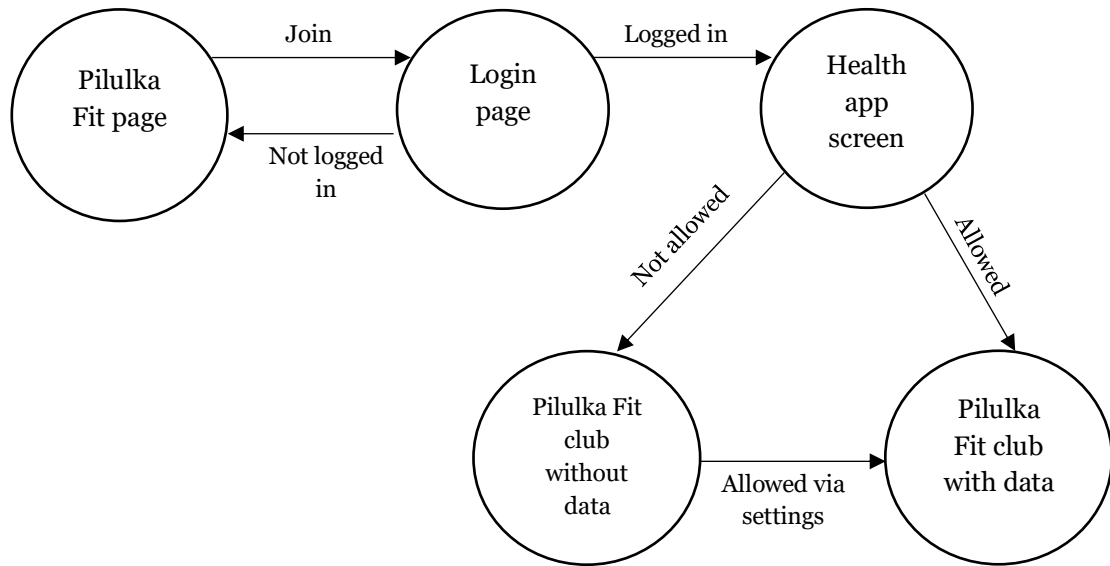
Conditions	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Did the user give a star rating?	F	F	F	F	F	F	F	F	T	T	T	T	T	T	T	T
Did the user write at least one positive point in the section "Klady a záporny"?	F	F	F	F	T	T	T	T	F	F	F	F	T	T	T	T
Did the user write at least one negative point in the section "Klady a záporny"?	F	F	T	T	F	F	T	T	F	F	T	T	F	F	T	T
Did the user write a review in the form of text?	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T
Result	F	F	F	F	F	F	F	F	F	T	T	T	T	T	T	T

Table 7 provides the complete test coverage of the product review with 16 different combinations of conditions. According to Table 7, the review can be sent by completing any of the 7 combinations. Based on this table, it was decided to create 6 test cases, including 5 positive test cases (highlighted in green) and one of the alternative test cases that checks that the product review cannot be submitted (highlighted in yellow).

5.3 Admission to the Pilulka Fit club

The test cases for the requirement Admission to the Pilulka Fit club **US-3** are designed using state transition testing (see 2.2.4). It is only possible to join Pilulka Fit if one has an account in the application. The user is first asked to log in if he is not already logged in. Then the application asks for permission to download his personal data from the Health application in order to synchronize the user's steps and kilometres. If the user agrees, the application opens the Pilulka Fit screen. If the user refuses to share his personal data, he will still get redirected to the Pilulka Fit screen and can give access to the Health application via the "Nastavení" settings later. Figure 22 demonstrates system transitions for a non-logged in user, starting from the Pilulka Fit page.

Figure 22 State transitioning – Admission to the Pilulka Fit club [Source: own elaboration]



From the graphical representation in Figure 22, it is visible which application transitions should be tested to ensure the successful inclusion of a user in the Pilulka Fit club, including two different options for synchronising data. To make these state transitions clearer, Table 8 supplements each of the start states with an event that triggers a transition, the effect and the end state that was caused by the effect.

Table 8 Table of state transitioning [Source: own elaboration]

Start state	Event	Effect	End state
Pilulka Fit page	Join Pilulka Fit club	Application asks for a log in	Login page
Login page	Not logging in	User is not logged in	Pilulka Fit page
Login page	Logging in	User is logged in	Health app screen
Health app screen	User allowed to download the data	Opened Pilulka Fit club with synchronizing data	Pilulka Fit club with data
Health app screen	User did not allow to download the data	Opened Pilulka Fit club without synchronizing data	Pilulka Fit club without data
Pilulka Fit club without data	User allowed to synchronize data via settings	Opened Pilulka Fit club with synchronizing data	Pilulka Fit club with data

6 Test case design based on user stories

This chapter is dedicated to **the design of test cases for the mobile application Pilulka directly related to the Subsection 4.3**. The user stories and their acceptance criteria described in Table 4 serve as an excellent basis for the design of test cases. Each test case presented in this work has the same structure. In the header, the test case contains its own **ID**, **name** and **preconditions**. The header section is then followed by 3 columns. The **Step number** column gives the number of the particular step. The next column **Step description** contains the description and the actions a tester has to perform to execute a test case. The last column, called the **Expected result**, represents the expected behaviour of the application, which the tester can use to determine the outcome of a test by comparing the actual results with the expected results.

6.1 Login

The **US-4 Login** user story specifies functionality related to the login process, including 4 different methods of logging into the application. The test cases developed in this subsection verify the login functionality via email, AppleID, Google account, and Facebook. **TC_04.01** including negative and positive testing of login via email and password. As a result of switching between login methods, the logout function is also being tested in **TC_04.02**.

Table 9 TC_04.01 – Login via email [Source: own elaboration]

Test Case ID and name	TC_04.01 – Login via email	
Preconditions	• Non-logged in user • Start from the profile screen	
Step number	Step description	Expected result
1	Write invalid email address	Error message is displayed
2	Write invalid password and log in	Error message is displayed
3	Write valid email address, invalid password, and log in	Error message is displayed
4	Click on “Přihlasit se” without filling in password and email address	Application asks to fill in the email and password
5	Write valid email address and log in	Application asks to fill in the password
6	Write valid password and log in	Application asks to fill in the email

7	Write valid email address and password	User is logged in
---	--	-------------------

Table 10 TC_04.02 – Login via Apple ID, Google Account and Facebook [Source: own elaboration]

Test Case ID and name	TC_04.02 – Login via Apple ID, Google Account and Facebook	
Preconditions	• Non-logged in user • Start from the profile screen	
Step number	Step description	Expected result
1	Click on AppleID icon	Sign in with Touch ID is displayed
2	Log in via Touch ID	User is logged in
3	Click on “Odhlásit se”	User is not logged in
4	Click on Google Account icon	Google window for login is opened
5	Log in with Google form	User is logged in
6	Click on “Odhlásit se”	User is not logged in
7	Click on Facebook icon	Login page via Facebook is opened
8	Log in via Facebook page	User is logged in
9	Click on “Odhlásit se”	User is not logged in

6.2 Favourite products

Test cases below are designed according to the feature described in **US-5 Favourite products**. **TC_05.01** contains test steps that ensure the correct behavior of this feature for non-logged in users. While **TC_05.02** deals with test steps from a perspective of a logged in user, who can mark/unmark a product as a favourite in different parts of the application like homepage, opened product or profile section.

Table 11 TC_05.01 – Favourite products for non-logged in users [Source: own elaboration]

Test Case ID and name	TC_05.01 – Favourite products for non-logged in users	
Preconditions	• Non-logged in user • Start from the profile screen	
Step number	Step description	Expected result
1	Check that the products shown on the homepage do not have a heart mark	Products are shown without heart marks for non-logged in users
2	After clicking on a random product check that it does not have a heart mark	It is impossible to mark product as a favorite as a non-logged in user

Table 12 TC_05.02 – Favourite products for logged-in users [Source: own elaboration]

Test Case ID and name	TC_05.02 – Favourite products for logged-in users	
Preconditions	• Logged-in user • Start from the homepage screen	
Step number	Step description	Expected result
1	Check that the products shown on the homepage have a heart mark	Products are shown with heart marks for logged in users
2	Click on the heart mark of a random product	Heart becomes pink after clicking on it
3	Unmark the product	Heart becomes uncolored
4	Search for a product and open it	Product is displayed
5	Check that the opened product has a heart mark and click on it	Heart becomes pink after clicking on it
6	Check that “Oblíbené produkty” are shown at the profile section	“Oblíbené produkty” section is displayed
7	Check that the product that was marked in step 8 is in the list	Product that is marked as favourite is displayed in the list
8	Unmark the favourite product	Product has been removed from the list
9	Search the unmarked product and check that the heart is unmarked	Product is unmarked as a favourite product

6.3 Repeat order

The repeat order function based on **US-6 Repeat order** is tested with 2 test cases **TC_06.01** and **TC_06.02**. Depending on whether the shopping cart is empty or already contains some items, this may affect the behaviour of the application after clicking the repeat order button.

Table 13 TC_06.01 – Repeat order with an empty cart [Source: own elaboration]

Test Case ID and name	TC_06.01 – Repeat order with an empty cart	
Preconditions	• Logged-in user with previously made purchases • Empty shopping cart • Start from the profile screen	
Step number	Step description	Expected result
1	Check that orders have “ <i>Zopakovat nákup</i> ” button in the section of orders	Each of the already made purchases has “ <i>Zopakovat nákup</i> ” button
2	Repeat order with only one product	Info message of changes in the cart is displayed
3	Check that the product is in the cart	Product from the repeated order is in the cart
4	Repeat order with more than one product	Info message of changes in the cart is displayed
5	Check that in addition to the previously added product there are new products	Products from the repeated orders are in the cart
6	Remove all the products from the card	Products can be removed

Table 14 TC_06.02 – Repeat order not with an empty cart [Source: own elaboration]

Test Case ID and name	TC_06.02 – Repeat order not with an empty cart	
Preconditions	• Logged-in user with previously made purchases • Shopping cart with more than 2 items in it • Start from the profile screen	
Step number	Step description	Expected result
1	Check that orders have “ <i>Zopakovat nákup</i> ” button in the section of orders	Each of the already made purchases has “ <i>Zopakovat nákup</i> ” button
2	Repeat order with more than one product	Info message of changes in the cart is displayed

3	Check that items that were in the cart before are not removed	Items are not removed
4	Check that in addition to the items that were already in the cart there are new products	Products from the repeated orders are in the cart
5	Remove all the products from the cart	Products can be removed

6.4 EAN scanner

This subsection presents a test case **TC_07.01** that defines the full coverage of EAN scanning within the application. Based on the acceptance criteria of the **US-7 EAN scanner**, this test case verifies whether the user can scan and find products with the camera of the phone, including the flashlight option.

Table 15 TC_07.01 – EAN scanning [Source: own elaboration]

Test Case ID and name	TC_07.01 – EAN scanning	
Preconditions	• Phone with a functioning camera • Product with EAN code • Start from the search screen	
Step number	Step description	Expected result
1	Click on the scanner icon	The application asks for the permission to use the phone's camera
2	Allow application to use the camera	The window for scanning the EAN code is displayed
3	Click on " <i>Blesk</i> "	The flashlight is on
4	Scan the product	Product is found
5	Click on the scanner icon	The window for scanning the EAN code is displayed
6	Scan the product without flashlight	Product is found
7	Click on the scanner icon	The window for scanning the EAN code is displayed
8	Click " <i>Zavřít</i> "	Window is closed

6.5 Substitution of products

User story **US-8 Substitution of products** contains of specific requirements about the substitution of non-available and out-of-stock products. Test cases **TC_08.01** and **TC_08.02** introduced in this subsection verify separately the functioning of this feature. Described steps test the ability to add non-available and out-of-stock products to the shopping cart, replace them with similar products or delete them.

Table 16 TC_08.01 – Substitution of non-available products [Source: own elaboration]

Test Case ID and name	TC_08.01 – Substitution of non-available products	
Preconditions	Found non-available products	
Step number	Step description	Expected result
1	Add a non-available product to the shopping cart	A non-available product is in the red frame with an info message around it
2	Click “Nahradit”	The pop-up window with substitute products is opened
3	Scroll the products	Products can be scrolled
4	Choose any product and click “Nahradit”	Info message about changes in the cart is displayed
5	Check that the product in the cart was replaced	Product is replaced with an available product
6	Add 2 non-available products to the shopping cart	2 non-available products are in the red frame with an info message around it
7	Click “Odstranit”	One of the products is removed
8	Click “Odstranit produkt” in the pop-up window of another product	Product is removed

Table 17 TC_08.02 – Substitution of out-of-stock products [Source: own elaboration]

Test Case ID and name	TC_08.02 – Substitution of out-of-stock products	
Preconditions	Found out-of-stock products	
Step number	Step description	Expected result
1	Add an out-of-stock product to the shopping cart	An out-of-stock product is in the yellow frame with an info message around it

2	Click “ <i>Nahradiť</i> ”	The pop-up window with substitute products is opened
3	Scroll the products	Products can be scrolled
4	Choose any product and click “ <i>Nahradiť</i> ”	Info message about changes in the cart is displayed
5	Check that the product in the cart was replaced	Product is replaced with an available product
6	Add 2 out-of-stock products to the shopping cart	2 out-of-stock products are in the yellow frame with an info message around it
7	Click “ <i>Odstrániť produkt</i> ” in the pop-up window of one product	Product is removed
8	Add non-available product to the shopping cart	A non-available product is in the red frame with an info message around it
9	Check that both products have their own frames around them	2 products have different frames and different info messages

6.6 Country of the application

The country change test in accordance with **US-9 Country of the application** can be done in 2 parts. Referring to the acceptance criteria, the country and the language of the application should be set automatically after the installation of the application. Besides that, the user can lately switch between 3 different countries in the settings: Czech Republic (CZ), Slovakia (SK) and Romania (RO). Therefore, it was decided to create **TC_09.01** which checks the dependency of the language of the application on the region settings and **TC_09.02** that verifies manual switch between countries.

Table 18 TC_09.01 – Automatic set of the country [Source: own elaboration]

Test Case ID and name	TC_09.01 – Automatic set of the country	
Preconditions	• Access to the region settings of the phone • Not installed application	
Step number	Step description	Expected result
1	Change the phone's region to Czech Republic	The region of the phone is set to Czech Republic
2	Download the application Pilulka	Application is downloaded
3	Open the application	Application is displayed in Czech language

4	Delete the application	Application is deleted
5	Change the phone's region to Slovakia	The region of the phone is set to Slovakia
6	Download the application Pilulka	Application is downloaded
7	Open the application	Application is displayed in Slovak language
8	Delete the application	Application is deleted
9	Change the phone's region to Romania	The region of the phone is set to Romania
10	Download the application Pilulka	Application is downloaded
11	Open the application	Application is displayed in Romanian language
12	Delete the application	Application is deleted
13	Change the phone's region to any other region	The region of the phone is set
14	Download the application Pilulka	Application is downloaded
15	Open the application	First screen that is displayed offers to select the country of the application: CZ, SK, RO
16	Click on "Česká republika"	The application is displayed in Czech language

Table 19 TC_09.02 – Country change via settings [Source: own elaboration]

Test Case ID and name	TC_09.02 – Country change via settings	
Preconditions	- Recently installed application - Non-logged in user - Start from the settings screen	
Step number	Step description	Expected result
1	From Czech version click on "Slovenská republika"	The country of the application is Slovakia, the application is in Slovak language
2	From Czech version click on "Romania"	The country of the application is Romania, the application is in Romanian language

3	From Slovak version click on “ <i>Česká republika</i> ”	The country of the application is Czech Republic, the application is in Czech language
4	From Slovak version click on “ <i>Romania</i> ”	The country of the application is Romania, the application is in Romanian language
5	From Romanian version click on “ <i>Česká republika</i> ”	The country of the application is Czech Republic, the application is in Czech language
6	From Romanian version click on “ <i>Slovenská republika</i> ”	The country of the application is Slovakia, the application is in Slovak language

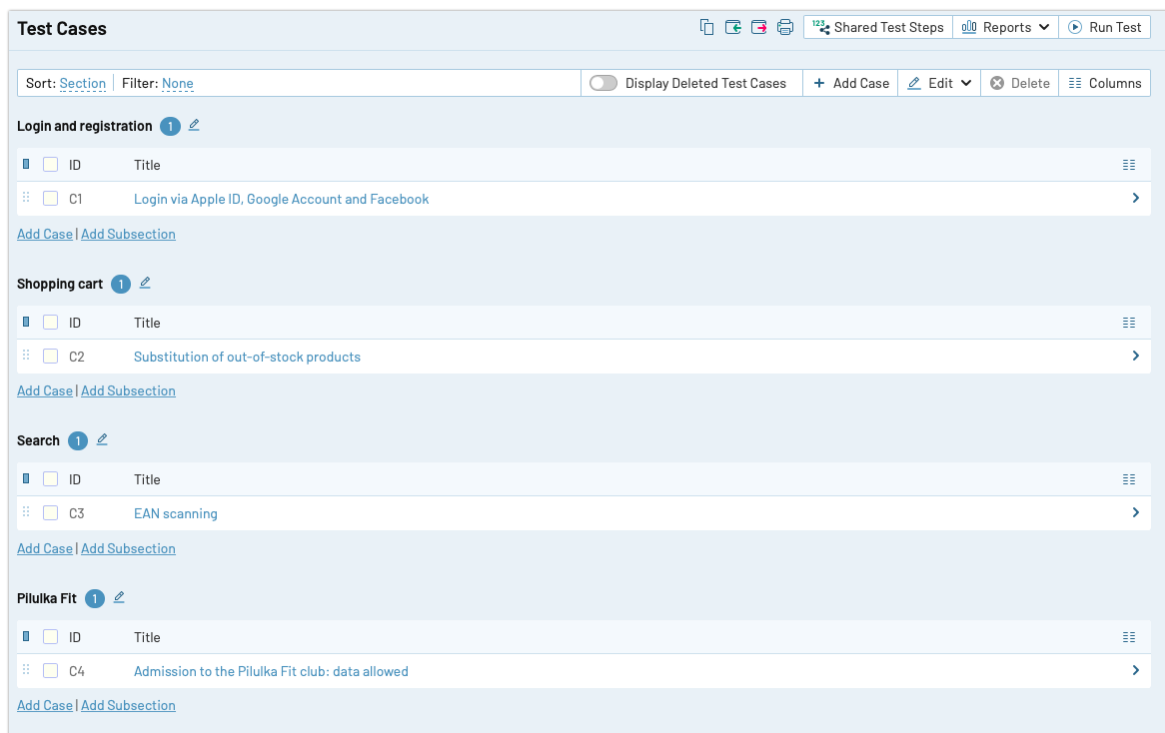
7 Management of test cases in TestRail

Undoubtedly, the whole process of creating, organizing, and updating test cases should be easy to manage and control. For example, as requirements change and application functionality is expanded, the number of test cases increases, as do the changes within the test steps. This suggests that the more test cases the testers write, the more difficult it is to control and renew them. A test management tool like TestRail (see 3.1) is a convenient alternative to archaic spreadsheets and text documents, allowing testers to keep track of all test cases quickly and efficiently. Therefore, this chapter **demonstrates the practical use of TestRail for managing the testing process** with the test cases designed in Chapter 5 and Chapter 6.

7.1 Test organization

To illustrate the ability of organising tests within TestRail by sections it was decided to import 4 test cases from 4 different sections of the Pilulka application: **TC_04.02** – Login via Apple ID, Google Account and Facebook, **TC_08.02** – Substitution of out-of-stock products, **TC_07.01** – EAN scanning and **TC_03.01** – Admission to the Pilulka Fit club: data allowed. As can be seen in Figure 23, each test case is located within its own section: Login and registration, Shopping cart, Search, Pilulka Fit. Division by such sections and eventually the sub-sections for specific features of the application makes it easier to orienteer between test cases, edit and use them.

Figure 23 Imported test cases [Source: own elaboration]



When editing test cases in TestRail, it is necessary to select mandatory attributes (see 3.1). For instance, the template of a test case named **C3 - EAN scanning**, was chosen as a template with steps, which means each of the steps is specified and provided with the expected result. From many options, the type of the test was marked as functional, and the priority of the test was set to medium. After entering the preconditions, the test steps, the expected results and saving the test, the test case **C3 - EAN scanning** is seen on the test cases page under the "Search" section.

Figure 24 The details of a test case: C3 - EAN scanning [Source: own elaboration]

The screenshot displays the TestRail interface for editing a test case titled "C3 - EAN scanning". The interface includes a header with the test case ID and name, and a navigation bar with back and forward arrows. The main form contains several sections:

- Title ***: A text field containing "EAN scanning".
- Section ***: A dropdown menu set to "Search".
- Template ***: A dropdown menu set to "Test Case (Steps)".
- Type ***: A dropdown menu set to "Functional".
- Priority ***: A dropdown menu set to "Medium".
- Estimate**: A text field.
- References**: A text field.
- Automation Type**: A dropdown menu set to "Select an Option".

Below the form fields is a **Preconditions** section with a list of conditions:

- Phone with a functioning camera
- Product with EAN code
- Start from the search screen

A note below the preconditions states: "The preconditions of this test case. Reference other test cases with [C#](e.g. [C17])."

The **Steps** section contains two steps:

- Step 1**:
 - Action**: "Click on the scanner icon"
 - Expected Result**: "The application asks for the permission to use the phone's camera"
- Step 2**:
 - Action**: "Allow application to use the camera"
 - Expected Result**: "The window for scanning the EAN code is displayed"

At the bottom right of the steps section, there are buttons for "Share", "Import", and a set of icons for adding, deleting, and moving steps.

7.2 Test run and tests

The next step after creating test cases and group them by sections was to build a Test Run. All the imported test cases were included in the test run called "**Test Run 4.13.2022, Version 6.23**". The run consists of individual tests introduced in Subsection 7.1 such as: logging in to the application, EAN scanning of products, substitutions of products in the shopping cart, joining the Pilulka Fit club. Based on the name of the test run it verifies the production version of the application Pilulka.cz – 6.23. Consequently, 4 tests were executed and saved with updated statuses and actual results. Figure 25 illustrates the status of a test case called **C2 - Substitution of out-of-stock products** and that this test took 3 minutes in total to complete. Figure 26 shows that the individual steps were marked as "Passed" and verified with expected results.

Figure 25 Status of the test: C2 - Substitution of out-of-stock products [Source: own elaboration]

Add Result

Status*
Passed
Set the test status (passed, failed etc.).

Comment
As per expected

Assign To
Alesia Zhiliaeva
Assign to another team member.

Version
6.23
The version you tested against.

Elapsed
3min
How long the test took.

Defects
A list of IDs in your bug tracker.

Steps
1 Add an out-of-stock product to

✓ Add Result × Cancel

Figure 26 Statuses of test steps: C2 - Substitution of out-of-stock products [Source: own elaboration]

Add Result

Steps
1 Add an out-of-stock product to the shopping cart
Expected Result
An out-of-stock product is in the yellow frame with info message around it
Enter actual result

2 Click "Nahradit"
Expected Result
The pop-up window with substitute products is opened
Enter actual result

Defects
A list of IDs in your bug tracker.

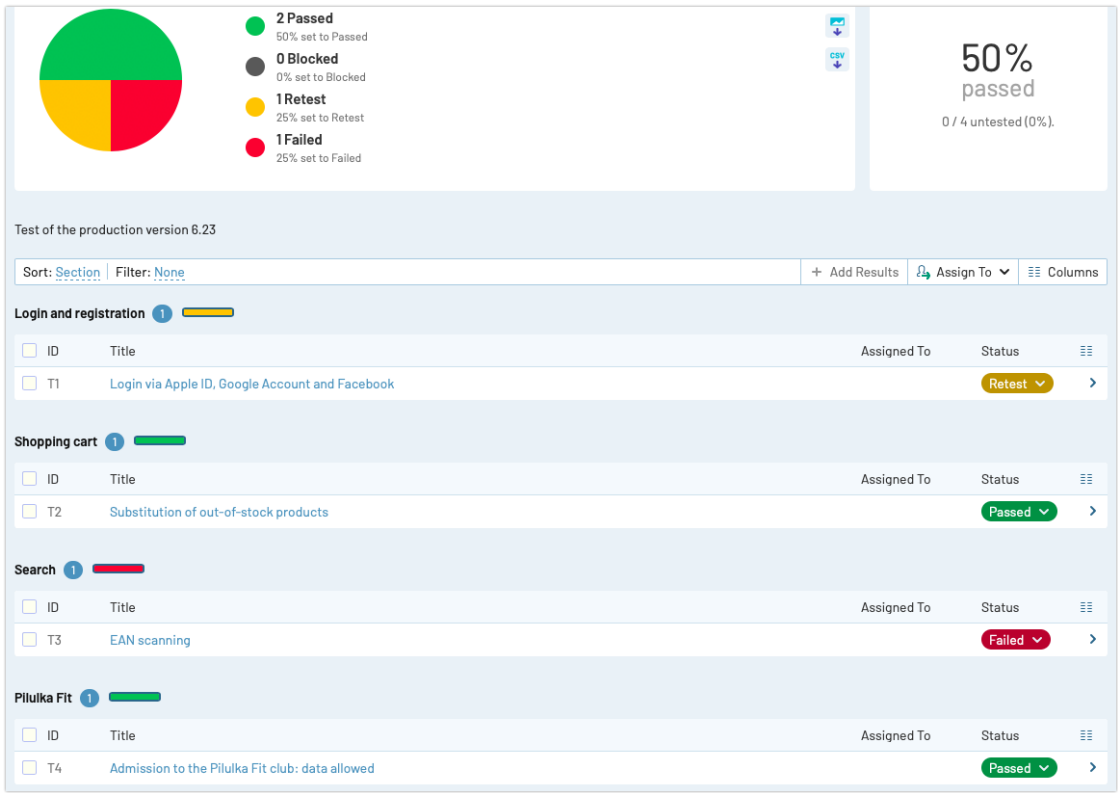
✓ Add Result × Cancel

7.3 Test results

The summary of the test run in Figure 27 shows the testing activity of completed tests. The overview includes the diagram and a percentage of tests with 4 statuses: Passed, Failed, Blocked and Retest. Based on the statistics calculated by TestRail it can be stated that only 50% of executed tests passed. It is also visible the number and names of tests that passed, failed blocked or set to retest. For instance, it was determined that the test case **C1 - Login via Apple ID, Google Account and Facebook** should be tested again. Due to some problems with Google Authentication, this test is marked as yellow and set to Retest. The test case **C3 - EAN scanning** makes 25% of failed tests (see Figure 27). The reason for that was that one of the test steps resulted in an error with not finding a scanned product. Test

cases **C2 - Substitution of out-of-stock products** and **C4 - Admission to the Pilulka Fit club: data allowed** are marked as passed.

Figure 27 The overview of the test run [Source: own elaboration]



Conclusion

The main aim of this work was to design test cases for the mobile application Pilulka by analyzing software requirements and applying complex testing techniques for designing test cases. This goal could only be reached by achieving the objectives of this thesis. The first objective was to introduce all the necessary theoretical knowledge, such as requirements elicitation, requirements specification, test case design techniques and test management tools. This objective was fulfilled in Chapter 1 "Software requirements", Chapter 2 "Test analysis methods" and Chapter 3 "Test management tools".

The next objective of this work was to select and summarize functional requirements of the application under test in order to design the test cases. By specifying the requirements in the form of user stories and defining their acceptance criteria. The third objective was to describe the specifics of the online pharmacy market. These two goals were achieved in Chapter 4 "Requirements analysis of the tested application".

The next objective was to design test cases based on the user stories presented in Chapter 4, using specification-based testing techniques and a specified test case format. This objective was fulfilled in Chapter 5 "Application of specification-based testing techniques", by applying test methods to the selected requirements and in Chapter 6 "Test case design based on user stories" by designing test cases based on functional requirements.

The last objective of this work was to give an illustrative example of working with test cases within the test management tool TestRail. It was accomplished by demonstrating the organization of tests, import process of chosen test cases and running them in the most widely used test management tool called TestRail. Chapter 7 was entirely dedicated to this objective.

The main contribution of this thesis is the created test cases for selected functional requirements of the mobile application Pilulka. The test cases designed in this work consist of detailed but not complex test steps, that can be executed by testers with different levels of experience. In case if the company decides to write automation tests for the Pilulka application, the overview of test cases presented in this thesis can help testers to determine which test cases should be automated.

List of references

AllTheResearch. (2021). *Online pharmacy market is expected to reach USD 202.3 billion by 2027*. NASDAQ OMX's News Release Distribution Channel. Retrieved March 10, 2022, from <https://www.proquest.com/wire-feeds/online-pharmacy-market-is-expected-reach-usd-202/docview/2570020315/se-2?accountid=17203>

App Annie. (2021). *The test of Mobile Application 2021*. Data.Ai. Retrieved March 20, 2022, from <https://www.data.ai/ru/go/state-of-mobile-2021/>

BARTLETT, J. (2022). *How to Write Test Cases for Software (with a sample)*. TestLodge Blog. Retrieved March 24, 2022, from <https://blog.testlodge.com/how-to-write-test-cases-for-software-with-sample/>

BLACK, R., Erik Van VEENENDAAL, GRAHAM, D., & Cengage Learning. (2017). *Foundations of software testing: ISTQB certification*. Boston: Cengage Learning Emea.

CHUNG, L., NIXON, B., A., Yu, E., & MYLOPOULOS, J. (2012). *Non-Functional Requirements in Software Engineering*. Amsterdam: Springer Science & Business Media.

Facts & Factors. (2022). *Online Pharmacy Market*. Retrieved April 11, 2022, from <https://www.globenewswire.com/news-release/2022/01/17/2367961/0/en/Global-Online-Pharmacy-Market-Size-To-Hit-USD-210-35-Billion-by-2028-at-a-17-83-CAGR-Growth-with-COVID-19-Analysis-Facts-Factors.html>

FITTLER, András, et al. *Consumers turning to the internet pharmacy market: cross-sectional study on the frequency and attitudes of Hungarian patients purchasing medications online*. Journal of medical internet research, 2018, 20.8: e11115.

GRADY, R. B., & CASWELL, D. L. (1987). *Software metrics: Establishing a company-wide program*. Prentice-Hall.

HAMBLING, B., MORGAN, P., SAMAROO, A., THOMPSON, G., & WILLIAMS, P. (2010). *Software testing: An ISTQB-ISEB Foundation guide*. London: BCS, The Chartered Institute.

HAMILTON, T. (2020, January 16). *25 BEST Test Management Tools*. Guru99. Retrieved April 13, 2022, from <https://www.guru99.com/top-20-test-management-tools.html>

HAMILTON, T. (2022, February 12). *Decision table testing: Learn with example*. Guru99. Retrieved April 13, 2022, from <https://www.guru99.com/decision-table-testing.html>

IEEE Computer Society. (1990). *IEEE Standard Glossary of Software Engineering Terminology*. New York: Institute of Electrical and Electronics Engineers, 1990. ISBN 9780738103914.

IEEE Computer Society. (2005). *IEEE standard for software verification and validation*. IEEE Std 1012-2004 (revision of IEEE Std 1012-1998). Rev. ed. New York, 2005. ISBN 0738146420.

ISTQB Glossary. (2021). Istqb.Org. Retrieved March 22, 2022, from <https://glossary.istqb.org/en/search/>

Jama Software. (2022, February 18). *Software Requirements and Software Requirements Specifications (SRS)*. Retrieved April 10, 2022, from <https://www.jamasoftware.com/requirements-management-guide/writing-requirements/the-characteristics-of-excellent-requirements>

KNOTT, D. (2015). *Hands-on mobile app testing: A guide for mobile testers and anyone involved in the mobile app business*. Boston: Addison-Wesley Professional.

RAJKUMAR. (2018, July 30). *Boundary value analysis test case design technique*. Retrieved March 4, 2022, from <https://www.softwaretestingmaterial.com/equivalence-partitioning-testing-technique/>

REJNKOVÁ, P. (n.d.). *MMSP copyright*. Vse.cz. Retrieved March 24, 2022, from https://spicenter.vse.cz/wpcontent/uploads/metodiky/publikovana_MMSP/MMSP/guidances/supportingmaterials/mmsp_copyright_3DC26695.html

ROBERTSON, S. & ROBERTSON, J. (2012). *Mastering the Requirements Process*. Boston: Addison-Wesley.

ROUDENSKÝ, P. a HAVLÍČKOVÁ, A. (2013). *Řízení kvality softwaru: průvodce testováním*. Brno: Computer Press.

TESTRAIL. (2020, February 17). *Our Story*. <https://www.gurock.com/testrail/about/>

SPELLNER, A., LINZ, T., & SCHAEFER, H. (2014). *Software testing foundations: A study guide for the certified tester exam (4th ed.)*. Santa Barbara: Rocky Nook.

STELLMAN, A., & GREENE, J. (2005). *Applied Software Project Management*. Sebastopol: O'Reilly Media, Inc.

WIEGERS, K., & BEATTY, J. (2013). *Software Requirements*. Redmond: Pearson Education.

Annexes

Annex A: Test case – Free delivery

Table 20 TC_01.01 – Free delivery: boundary values [Source: own elaboration]

Test Case ID and name	TC_01.01 – Free delivery: boundary values	
Preconditions	Found product/products with a total sum of 1998, 1999 and 2000 Kč	
Step number	Step description	Expected result
1	Add product/products to the shopping cart with a total amount of 1999 Kč	Product/products are in the shopping cart
2	Check that the info message at the top of the screen shows “ <i>Dopravu máte zdarma</i> ”	The info message is displayed
3	Click on delivery “ <i>Doprava</i> ”	All the delivery options are available for free
4	Add product/products to the shopping cart with a total amount of 2000 Kč	Product/products are in the shopping cart
5	Check that the info message at the top of the screen shows “ <i>Dopravu máte zdarma</i> ”	The info message is displayed
6	Click on delivery “ <i>Doprava</i> ”	All the delivery options are available for free
7	Remove all the items from the shopping cart	Shopping cart is empty
8	Add product/products to the shopping cart with a total amount of 1998 Kč	Product/products are in the shopping cart
9	Check that the scale indicating how much is left before free delivery is shown at the top of the screen	Scale indicating how much is left before free delivery is displayed
10	Click on delivery “ <i>Doprava</i> ”	Free delivery for all the delivery options is not available

Annex B: Test cases – Product review

Table 21 TC_02.01 – Submission of the product review: stars and text [Source: own elaboration]

Test Case ID and name	TC_02.01 – Submission of the product review: stars and text	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result
1	Do star rating	Stars are selected
2	Write text of the review	The field for text is filled in
3	Click on “ <i>Odeslat hodnocení</i> ”	Info message “ <i>Děkujeme za hodnocení</i> ” is displayed
4	Check that the sent review is visible in the “ <i>Hodnocení</i> ” section of the product	Review is shown

Table 22 TC_02.02 – Submission of the product review: stars and a positive point [Source: own elaboration]

Test Case ID and name	TC_02.02 – Submission of the product review: stars and a positive point	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result
1	Do star rating	Stars are selected
2	Write a positive point in the “ <i>Klady</i> ” section	Positive point is inserted
3	Click on “ <i>Odeslat hodnocení</i> ”	Info message “ <i>Děkujeme za hodnocení</i> ” is displayed
4	Check that the sent review is visible in the “ <i>Hodnocení</i> ” section of the product	Review is shown

Table 23 TC_02.03 – Submission of the product review: stars and a negative point [Source: own elaboration]

Test Case ID and name	TC_02.03 – Submission of the product review: stars and a negative point	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result
1	Do star rating	Stars are selected
2	Write a negative point in the “Zápory” section	Negative point is inserted
3	Click on “Odeslat hodnocení”	Info message “Děkujeme za hodnocení” is displayed
4	Check that the sent review is visible in the “Hodnocení” section of the product	Review is shown

Table 24 TC_02.04 – Submission of the product review: stars, negative and positive points [Source: own elaboration]

Test Case ID and name	TC_02.04 – Submission of the product review: stars, negative and positive points	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result
1	Do star rating	Stars are selected
2	Write a positive point in the “Klady” section	Positive point is inserted
3	Write a negative point in the “Zápory” section	Negative point is inserted
4	Click on “Odeslat hodnocení”	Info message “Děkujeme za hodnocení” is displayed

5	Check that the sent review is visible in the “ <i>Hodnocení</i> ” section of the product	Review is shown
---	--	-----------------

Table 25 TC_02.05 – Submission of the product review: stars, text, negative and positive points [Source: own elaboration]

Test Case ID and name	TC_02.05 – Submission of the product review: stars, text, negative and positive points	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result
1	Do star rating	Stars are selected
2	Write a positive point in the “ <i>Klady</i> ” section	Positive point is inserted
3	Write a negative point in the “ <i>Zápory</i> ” section	Negative point is inserted
4	Write text of the review	The field for text is filled in
5	Click on “ <i>Odeslat hodnocení</i> ”	Info message “ <i>Děkujeme za hodnocení</i> ” is displayed
6	Check that the sent review is visible in the “ <i>Hodnocení</i> ” section of the product	Review is shown

Table 26 TC_02.06 – Submission of the product review: no stars and a positive point [Source: own elaboration]

Test Case ID and name	TC_02.06 – Submission of the product review: no stars and a positive point	
Preconditions	• Logged-in user • Found product for leaving a review • Start from the review form of the product	
Step number	Step description	Expected result

1	Write a positive point in the “ <i>Klady</i> ” section	Positive point is inserted
2	Check that the button “ <i>Odeslat hodnocení</i> ” is grey	Button is grey, review cannot be sent

Annex C: Test cases – Admission to the Pilulka Fit club

Table 27 TC_03.01 – Admission to the Pilulka Fit club: data allowed [Source: own elaboration]

Test Case ID and name	TC_03.01 – Admission to the Pilulka Fit club: data allowed	
Preconditions	- Recently installed application - Non-logged in user - Start from the Pilulka Fit page	
Step number	Step description	Expected result
1	Agree with the conditions of Pilulka Fit service and click on “ <i>Chci žít zdravěji</i> ”	The application asks if the user wants to login
2	Choose “ <i>Ano</i> ”	Login page is opened
3	Login to the application	The application asks for access to the Health application
4	Allow Pilulka to read the Health data	Data is synchronized. The user is admitted to the club

Table 28 TC_03.02 – Admission to the Pilulka Fit club: data not allowed [Source: own elaboration]

Test Case ID and name	TC_03.02 – Admission to the Pilulka Fit club: data not allowed	
Preconditions	- Recently installed application - Non-logged in user - Start from the Pilulka Fit page	
Step number	Step description	Expected result
1	Agree with the conditions of Pilulka Fit service and click on “ <i>Chci žít zdravěji</i> ”	The application asks if the user wants to login

2	Choose “Ano”	Login page is opened
3	Login to the application	The application asks for access to the Health application
4	Do not allow Pilulka to read the Health data	Data is not synchronized. The user is admitted to the club

Table 29 TC_03.03 – Admission to the Pilulka Fit club: data allowed via settings [Source: own elaboration]

Test Case ID and name	TC_03.03 – Admission to the Pilulka Fit club: data allowed via settings	
Preconditions	• Recently installed application • Non-logged in user • Start from the Pilulka Fit page	
Step number	Step description	Expected result
1	Agree with the conditions of Pilulka Fit service and click on “ <i>Chci žít zdravěji</i> ”	The application asks if the user wants to login
2	Choose “Ano”	Login page is opened
3	Login to the application	The application asks for access to the Health application
4	Do not allow Pilulka to read the Health data	Data is not synchronized. The user is admitted to the club
5	Click on settings of Pilulka Fit	Settings page is displayed
6	Allow Pilulka to read the Health data through the Health application	Data is synchronized.

Table 30 TC_03.04 – Admission to the Pilulka Fit club: without logging in [Source: own elaboration]

Test Case ID and name	TC_03.04 – Admission to the Pilulka Fit club: without logging in	
Preconditions	• Recently installed application • Non-logged in user • Start from the Pilulka Fit page	

Step number	Step description	Expected result
1	Agree with the conditions of Pilulka Fit service and click on “ <i>Chci žít zdravěji</i> ”	The application asks if the user wants to login
2	Choose “Ne”	Initial Pilulka Fit page is displayed. The user is not admitted to the club